

Univerzitet u Kragujevcu
Fakultet inženjerskih nauka



Seminarski rad iz predmeta:
Bioinženjering i bioinformatika

Tema:
Program za generisanje 3D modela srca

Student:

Anita Mijatović 612/2022

Profesor:

dr Nenad Filipović, red. Prof

Kragujevac, 2026.

1. UVOD.....	3
2.1 Ulazni podaci.....	4
2.2 Učitavanje i organizacija podataka	5
2.3 Izazovi u obradi podataka	5
2.5 Kriterijumi uspeha.....	6
3. METODOLOGIJA RADA	7
3.1 Okruženje i biblioteke	7
pydicom	8
NumPy	8
SimpleITK	9
scikit-image	9
SciPy	9
matplotlib.....	9
PyVista.....	10
Instalacija i konfiguracija okruženja.....	10
Provera instalacije.....	10
Značaj izbora okruženja.....	11
3.2 Organizacija koda, konfiguracija i upravljanje putanjama(uvoz biblioteka)	11
3.3. Učitavanje podataka i priprema volumena	17
3.4. Robusna HU normalizacija	20
3.5. Izotropno resamplovanje	21
3.6. Brzi informativni ispis (volumen, HU opseg, prag, maska).....	21
3.7. Multiplanarna vizuelizacija i validacija segmentacije	22
3.8. Analiza osetljivosti HU pragova na kvalitet segmentacije.....	23
4.REZULTATI.....	39
4.1.Izgled histograma	40
4.2.Uticaj HU pragova na segmentaciju.....	40

4.3.Pregled preseka	41
4.4.Aksijalni preseci	42
4.5. Trodimenzionalna rekonstrukcija i volumetrijska analiza	43
4.6.3D rekonstrukcija i renderi.....	44
5.ZAKLJUCAK.....	46
6.LITERATURA	48

1. UVOD

Razvoj savremenih tehnologija u oblasti medicinske dijagnostike i računarske obrade podataka omogućio je značajan napredak u vizualizaciji anatomskih struktura ljudskog tela. Jedna od najvažnijih primena ovih tehnologija jeste 3D modelovanje organa, među kojima posebno mesto zauzima srce kao centralni organ kardiovaskularnog sistema. 3D modelovanje srca predstavlja proces kreiranja trodimenzionalne digitalne reprezentacije srčane anatomije, koja omogućava detaljniji uvid u njegovu strukturu u odnosu na tradicionalne dvodimenzionalne prikaze.

Značaj 3D modelovanja srca ogleda se u njegovoj primeni u hirurškoj pripremi, edukaciji i dijagnostici. Trodimenzionalni modeli omogućavaju lekarima preciznije planiranje operacija i smanjenje rizika od komplikacija, dok studentima i stručnjacima olakšavaju razumevanje kompleksne anatomije srca. Takođe, u dijagnostici omogućavaju lakše uočavanje patoloških promena koje mogu biti teže vidljive na standardnim snimcima.

Primena 3D modelovanja obuhvata planiranje operacija, analizu srčanih mana i razvoj personalizovane medicine. Generisanjem modela na osnovu podataka specifičnih za pacijenta omogućava se individualizovan pristup lečenju, posebno kod složenih anatomskih slučajeva. Pored toga, ovi modeli se koriste i u komunikaciji sa pacijentima radi lakšeg objašnjenja medicinskih procedura.

Razvoj 3D rekonstrukcije započeo je uvođenjem CT i MRI tehnologija, dok su savremeni algoritmi i povećana procesorska snaga omogućili generisanje preciznih trodimenzionalnih modela. Danas softverski alati omogućavaju automatsku ili poluautomatsku segmentaciju i rekonstrukciju anatomskih struktura, čime je ovaj proces značajno unapređen.

Cilj ovog rada jeste razvoj programa za automatsko generisanje 3D modela srca na osnovu CT snimaka. Proces obuhvata obradu medicinskih slika, segmentaciju srčane strukture i rekonstrukciju trodimenzionalnog modela koji verno predstavlja anatomiju srca, sa ciljem demonstracije primene savremenih računarskih tehnika u medicini.

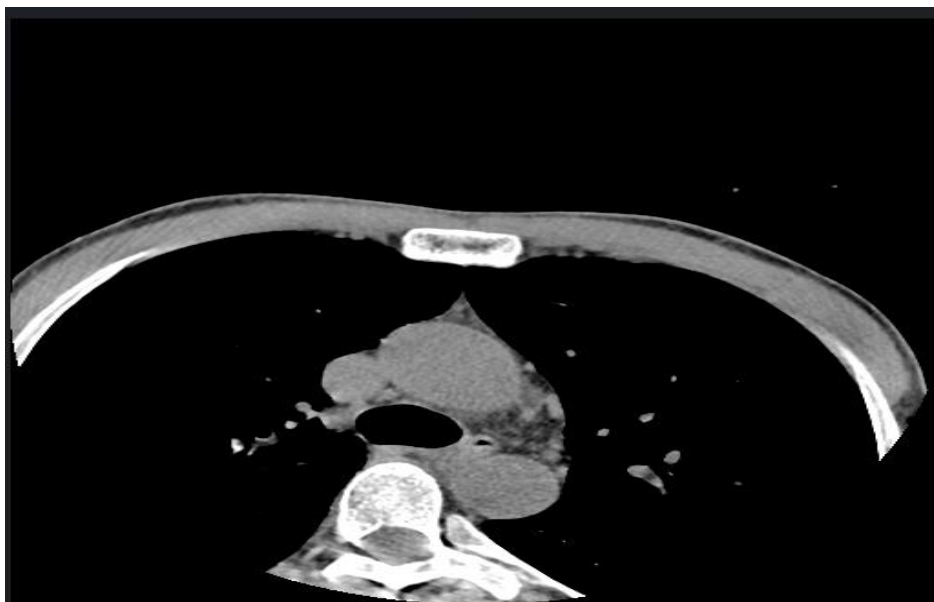
U okviru savremenih pristupa 3D rekonstrukciji srca, posebna pažnja posvećuje se i segmentaciji koronarnih arterija, koje predstavljaju ključne krvne sudove odgovorne za snabdevanje srčanog mišića kiseonikom. Njihova vizualizacija u trodimenzionalnom prostoru omogućava detaljniju analizu njihove strukture, grananja i eventualnih suženja, što je od velikog značaja u dijagnostici kardiovaskularnih oboljenja. Integracijom rekonstrukcije srca i izdvajanja koronarnih arterija postiže se potpuniji i precizniji prikaz anatomske celine, čime se dodatno unapređuje primena 3D modelovanja u medicinske svrhe, posebno u planiranju intervencija i analizi patoloških promena.

2. OPIS PROBLEMA

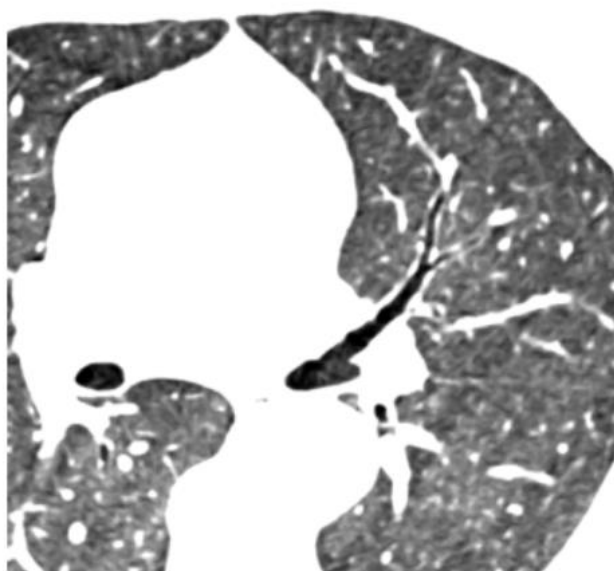
U okviru ovog rada razmatra se problem automatskog generisanja trodimenzionalnog modela srca na osnovu medicinskih CT snimaka. Ovaj problem obuhvata više faza obrade podataka, uključujući učitavanje medicinskih slika, njihovu analizu, segmentaciju željene anatomske strukture i konačno formiranje 3D modela.

Osnovni izazov u ovom procesu predstavlja činjenica da su ulazni podaci dati u formi dvodimenzionalnih preseka (slice-ova), koje je potrebno pravilno interpretirati i kombinovati kako bi se dobila verna trodimenzionalna reprezentacija srca. Pored toga, medicinski snimci sadrže različite tipove tkiva i šum, što dodatno otežava izdvajanje relevantnih struktura.

Poseban izazov predstavlja i segmentacija koronarnih arterija, koje su znatno manje dimenzije u odnosu na ostatak srčane strukture i često slabije izražene na CT snimcima. Njihovo precizno izdvajanje zahteva primenu dodatnih tehnika obrade slike, kao što su filtriranje, podešavanje pragova i izdvajanje finih struktura. Zbog svoje složene geometrije i grananja, koronarne arterije predstavljaju zahtevniji deo procesa rekonstrukcije, ali njihovo uključivanje omogućava potpuniji i detaljniji prikaz srca.



Slika 1- Primer ulaznog CT sloja citavo srce(DICOM)



Slika 2-Dicom snimak za koronarnu arteriju

2.1 Ulazni podaci

Ulazni podaci za generisanje 3D modela srca predstavljeni su u formatu DICOM (Digital Imaging and Communications in Medicine), koji predstavlja standard u medicinskoj dijagnostici za skladištenje i razmenu medicinskih slika. DICOM fajlovi sadrže i metapodatke kao što su dimenzije slike i razmak između slojeva (spacing), koji su važni za rekonstrukciju.

Kragujevac,2026.

Tipičan CT dataset srca sastoji se od više uzastopnih preseka (slice-ova), rezolucije približno 512×512 piksela. Ovi preseki predstavljaju tanke slojeve tela koji se koriste za formiranje trodimenzionalne strukture srca. Poseban izazov predstavlja izdvajanje koronarnih arterija, zbog njihove male dimenzije i složene strukture u odnosu na okolno tkivo.

Prostorna rezolucija (razmak između piksela i slojeva) direktno utiče na kvalitet 3D modela. U XY ravni iznosi približno 0.5–1 mm, dok je u Z osi 0.5–2 mm, što je posebno značajno za preciznu rekonstrukciju finih struktura kao što su koronarne arterije.

Svaki piksel u CT slici ima vrednost u Hounsfield (HU) skali, koja omogućava razlikovanje tkiva i segmentaciju struktura od interesa. Na osnovu ovih vrednosti moguće je izdvojiti srčani mišić i koronarne krvne sudove.

2.2 Učitavanje i organizacija podataka

Ove vrednosti su ključne za proces segmentacije, jer omogućavaju izdvajanje srčanog tkiva iz ostatka slike.

Prvi korak u okviru razvijenog pipeline-a jeste učitavanje ulaznih podataka. U ovom radu, podaci za rekonstrukciju srca dati su u formi serije PNG slika visokog kvaliteta, koje su prethodno konvertovane iz DICOM formata radi jednostavnije obrade u Python okruženju, uz očuvanje informacija o prostornom razmaku (spacing). Za učitavanje i obradu ovih slika koristi se biblioteka `imageio`, pri čemu je neophodno obezbediti pravilno sortiranje fajlova kako bi se očuvao anatomski redosled preseka.

Sa druge strane, podaci za koronarne arterije dostupni su u originalnom DICOM formatu, što omogućava direktan rad sa medicinskim metapodacima i HU vrednostima, koje su posebno važne za preciznu segmentaciju vaskularnih struktura.

2.3 Izazovi u obradi podataka

Obrada CT podataka srca i koronarnih arterija nosi više izazova u okviru računarske analize medicinskih slika. Jedan od osnovnih problema predstavlja prisustvo šuma i artefakata koji nastaju tokom procesa snimanja, što može negativno uticati na tačnost segmentacije i otežati izdvajanje anatomske strukture.

Dodatni izazov predstavlja sličnost intenziteta različitih tkiva u CT snimcima. Srčani mišić, krv i okolna meka tkiva često imaju bliske Hounsfield (HU) vrednosti, što otežava njihovo jasno razdvajanje i može dovesti do grešaka u procesu segmentacije, posebno kod koronarnih arterija koje su tanke i slabo izražene strukture.

Takođe, značajan izazov predstavlja varijabilnost anatomije između različitih pacijenata, kao i razlike u kvalitetu i rezoluciji snimaka. Zbog toga je neophodno da algoritam za obradu bude dovoljno robustan i prilagodljiv kako bi omogućio stabilne rezultate u različitim slučajevima.

2.4 Cilj obrade podataka

Cilj obrade ulaznih podataka jeste priprema CT snimaka za dalju segmentaciju i rekonstrukciju 3D modela srca. Ovo podrazumeva:

- pravilno učitavanje i sortiranje DICOM fajlova
- ekstrakciju relevantnih metapodataka
- organizaciju podataka u 3D strukturu
- pripremu za primenu algoritama segmentacije

Uspešna realizacija ovih koraka predstavlja osnovu za dalju obradu i generisanje preciznog trodimenzionalnog modela srca.

2.5 Kriterijumi uspeha

Uspešnost procesa 3D rekonstrukcije srca ocenjuje se na osnovu kvaliteta dobijenog modela i njegove primenljivosti u daljoj analizi i vizualizaciji. Da bi rekonstrukcija bila uspešna, potrebno je da model zadovolji kriterijume geometrijske tačnosti, jasnoće struktura i tehničke ispravnosti.

Pre svega, rekonstruisani model mora jasno prikazivati osnovne anatomske strukture srca, uključujući četiri srčane komore, pri čemu njihova prostorna organizacija treba da odgovara realnoj anatomiji. Takođe je neophodno da srce bude pravilno odvojeno od okolnih tkiva, što zavisi od kvaliteta segmentacije i utiče na preciznost finalnog modela.

Važan kriterijum predstavlja i kvalitet površine modela, koja treba da bude glatka i kontinuirana, bez izraženih artefakata i nepravilnosti nastalih tokom obrade. Pored vizuelnog kvaliteta, model mora biti anatomski konzistentan u pogledu oblika i proporcija.

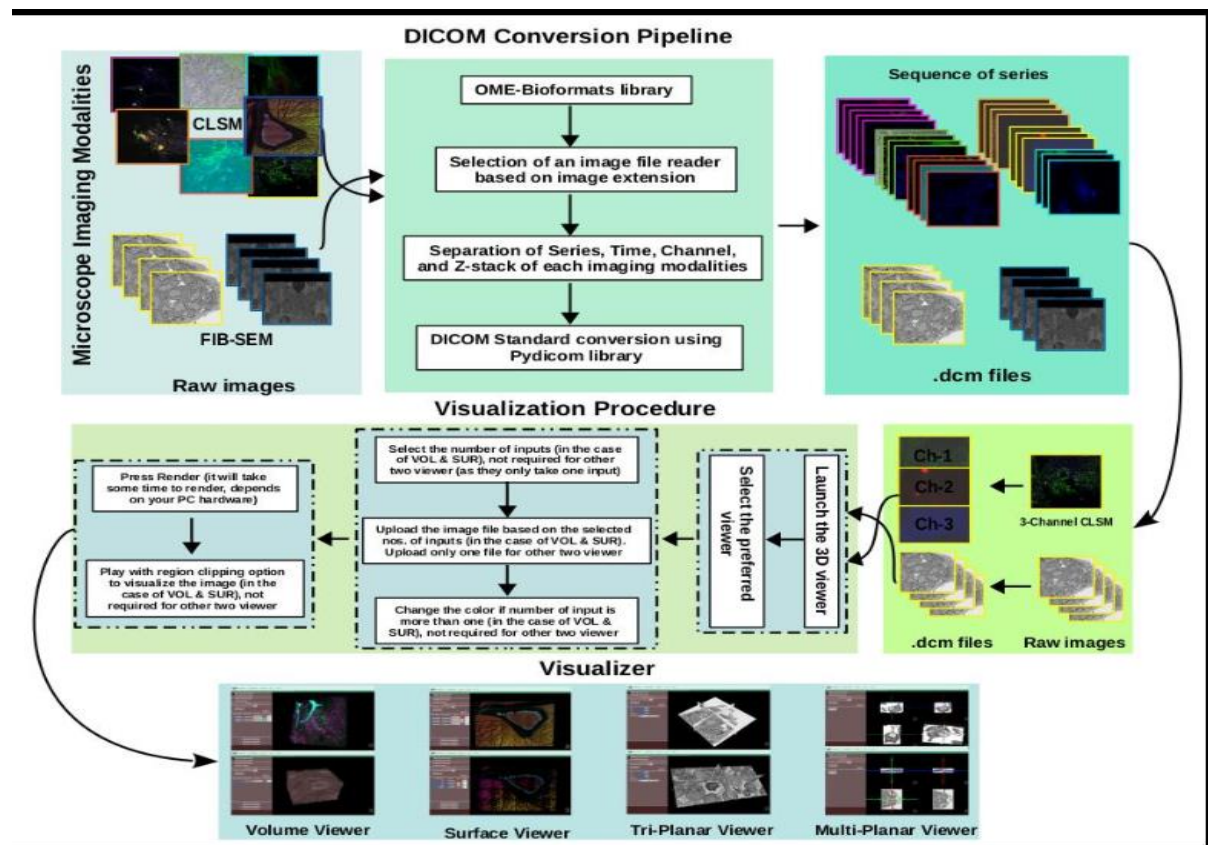
Na kraju, tehnička ispravnost podrazumeva mogućnost izvoza modela u standardne formate, kao što je STL, čime se omogućava njegova dalja upotreba u vizualizaciji i eventualnoj 3D štampi. Po potrebi, kvalitet rekonstrukcije može se procenjivati i kvantitativnim metrikama, poput Dice koeficijenta ili IoU, ukoliko postoji referentna segmentacija.

3. METODOLOGIJA RADA

3.1 Okruženje i biblioteke

U cilju realizacije sistema za automatsko generisanje 3D modela srca iz CT snimaka, korišćeno je programsko okruženje zasnovano na jeziku Python. Python je izabran zbog široke primene u oblasti naučnog računarstva, obrade medicinskih slika i razvoja algoritama, kao i velikog broja biblioteka za rad sa volumenskim podacima.

Razvoj je obavljen u Python 3.9 okruženju, u okviru Anaconda distribucije, koja obezbeđuje unapred konfigurisane pakete za naučno računarstvo. Kao razvojno okruženje korišćen je Spyder IDE, koji omogućava jednostavno testiranje koda i vizualizaciju rezultata, što je posebno značajno u obradi medicinskih slika.



Slika 3- Transformacija sirovih snimaka u DICOM

Korišćene biblioteke

Za implementaciju sistema korišćen je skup biblioteka koje pokrivaju različite aspekte obrade podataka – od učitavanja medicinskih slika, preko numeričke obrade, do generisanja i vizualizacije 3D modela.

pydicom

pydicom je biblioteka za rad sa DICOM fajlovima, standardom za medicinske slike. Omogućava učitavanje slika i pristup metapodacima kao što su dimenzije, rezolucija i informacije o snimanju. U ovom radu korišćena je za učitavanje CT preseka i njihovu pripremu za dalju obradu.

NumPy

NumPy je biblioteka za rad sa višedimenzionalnim nizovima podataka. U ovom radu koristi se za skladištenje volumenskih CT podataka, kao i za matematičke operacije, transformacije i normalizaciju piksela. Njena efikasnost omogućava obradu velikih skupova medicinskih podataka.

SimpleITK

SimpleITK je biblioteka za obradu medicinskih slika i volumenskih podataka. U ovom radu koristi se za filtriranje, poboljšanje kvaliteta slike i primenu osnovnih operacija nad CT podacima, čime se olakšava dalja segmentacija i rekonstrukcija.

scikit-image

scikit-image je biblioteka za obradu slika i implementaciju algoritama segmentacije. U ovom radu koristi se funkcija *marching cubes* za generisanje 3D površine iz volumenskih podataka, što predstavlja ključni korak u rekonstrukciji 3D modela iz medicinskih slika.

SciPy

SciPy je biblioteka koja se u ovom radu koristi za primenu morfoloških operacija nad slikama, uključujući eroziju, dilataciju i uklanjanje šuma. Ove operacije poboljšavaju kvalitet segmentacije i doprinose dobijanju preciznijeg 3D modela.

Matplotlib

Biblioteka za vizualizaciju podataka koja se u ovom radu koristi za prikaz pojedinačnih CT preseka i rezultata segmentacije, kao i za osnovnu analizu dobijenih rezultata. Omogućava vizuelnu proveru tačnosti obrade u različitim fazama pipeline-a.

PyVista

PyVista je biblioteka za 3D vizualizaciju i rendering. U ovom radu korišćena je za prikaz i interaktivnu manipulaciju 3D modela srca, kao i za vizualnu analizu dobijene rekonstrukcije.

Instalacija i konfiguracija okruženja

Za uspešno izvršavanje programa potrebno je instalirati sve navedene biblioteke. Instalacija se vrši putem Anaconda Prompt-a korišćenjem sledeće komande:

```
(base) C:\Users\Korisnik>pip install pydicom numpy SimpleITK scikit-image scipy matplotlib pyvista
```

Alternativno, moguće je koristiti i conda menadžer paketa:

```
(base) C:\Users\Korisnik>conda install -c conda-forge pydicom numpy simpleitk scikit-image scipy matplotlib pyvista
```

Provera instalacije

Nakon instalacije, neophodno je proveriti da li su biblioteke uspešno instalirane i dostupne u radnom okruženju. To se može uraditi pokretanjem sledećeg koda:

```
import pydicom
import numpy
import SimpleITK
import skimage
import scipy
import matplotlib
import pyvista

print(f"pydicom: {pydicom.__version__}")
print(f"numpy: {numpy.__version__}")
print(f"SimpleITK: {SimpleITK.Version.VersionString()}")
print(f"scikit-image: {skimage.__version__}")
print(f"scipy: {scipy.__version__}")
print(f"matplotlib: {matplotlib.__version__}")
print(f"pyvista: {pyvista.__version__}")
```

```
pydicom: 3.0.2
numpy: 1.26.4
SimpleITK: 2.5.3
scikit-image: 0.24.0
scipy: 1.13.1
matplotlib: 3.9.2
pyvista: 0.47.2
```

Značaj izbora okruženja

Izbor odgovarajućeg programskog okruženja i biblioteka ima ključnu ulogu u efikasnosti i kvalitetu implementacije. Kombinacija Python jezika i korišćenih biblioteka omogućava jednostavnu implementaciju složenih algoritama, efikasnu obradu velikih količina podataka, kao i kvalitetnu vizualizaciju rezultata.

Takođe, ovakav pristup obezbeđuje proširivost sistema i mogućnost daljih unapređenja. Zahvaljujući navedenim karakteristikama, razvijeni sistem predstavlja stabilnu osnovu za primenu u medicinskoj analizi i dalji razvoj.

3.2 Organizacija koda, konfiguracija i upravljanje putanjama(uvoz biblioteka)

Nakon definisanja osnovnog seta biblioteka, implementacija sistema oslanja se na strogo strukturiran Python kod koji obezbeđuje automatizaciju i modularnost čitavog procesa. Centralni deo ove faze predstavlja uspostavljanje hijerarhije direktorijuma i parametarskog upravljanja putem klase `Config`, čime se postiže visoka fleksibilnost aplikacije. Ovakav pristup omogućava da se izmenom samo nekoliko linija koda sistem prilagodi različitim setovima CT snimaka, bez potrebe za intervencijom u samoj logici algoritama.

```
import os
import sys
import time
import numpy as np
import imageio.v2 as imageio
from skimage import measure, filters, morphology, exposure
from skimage.filters import gaussian
from skimage.morphology import binary_closing, binary_opening, ball, disk, remove_small_objects
from scipy.ndimage import binary_fill_holes, zoom, binary_erosion, binary_dilation
import pyvista as pv
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
import warnings
warnings.filterwarnings('ignore')
```

Slika 4 –Uvoz svih potrebnih biblioteka

Implementacija biblioteka je podeljena na dva modula u zavisnosti od izvora podataka. Za rad sa **PNG slikama uvezenim iz DICOM formata**, korišćen je pun skup biblioteka, uključujući `imageio` za efikasno učitavanje i manipulaciju slikovnim podacima, uz `scikit-image`, `scipy`, `numpy` i `pyvista` za dalju obradu i 3D vizuelizaciju. Nasuprot tome, za direktnu analizu **DICOM podataka koronarnih arterija**, korišćene su sve navedene biblioteke osim `imageio`, s obzirom na to da se proces uvoza vrši putem specijalizovanih alata za medicinsko snimanje, a ne putem obrade slikovnih fajlova.

```
try:
    import pydicom
    PYDICOM_AVAILABLE = True
except ImportError:
    PYDICOM_AVAILABLE = False
    print("⚠ UPOZORENJE: pydicom nije instaliran!")
    print("  Instaliraj ga komandom: pip install pydicom")
    print("  Program će pokušati da nastavi bez DICOM podrške.\n")
```

Slika 5- Implementacija bezbednog uvoza biblioteke pydicom radi osiguranja stabilnosti izvršavanja koda za dicom strukturu

```

class Config:
    """Centralizovana konfiguracija za sve parametre"""

    INPUT_DIR = r"C:\Users\Korisnik\Downloads\archive (1)\DICOM TO PNG"
    OUTPUT_DIR = r"C:\Users\Korisnik\Downloads\archive (1)\heart_3d_output"

    HU_MIN = -500
    HU_MAX = 1000

    HEART_HU_LOWER = 40
    HEART_HU_UPPER = 300

    SPACING_MM = (0.8, 0.8, 1.0)

    GAUSSIAN_SIGMA = 1.0

    CLOSING_RADIUS = 2
    MIN_OBJECT_SIZE = 5000

    MARCHING_CUBES_LEVEL = 0.5
    SMOOTHING_ITERATIONS = 500
    SMOOTHING_RELAXATION = 0.05

    RENDER_RESOLUTION = [1920, 1080]
    MESH_COLOR = 'maroon'
    BACKGROUND_COLOR = 'white'

config = Config()

```

Slika 6-definisanje klase Config i upravljanje putanjama

U okviru konfiguracije, putanje do izvornih PNG slika i izlaznog direktorijuma definisane su apsolutnim putanjama, čime se eliminiše rizik od grešaka pri izvršavanju skripte iz različitih radnih okruženja. Pored organizacije fajlova, klasa `Config` integriše i ključne fizičke parametre, kao što su prostorna rezolucija snimka (*spacing*) i precizno definisani pragovi na Hounsfieldovoj skali. Ovi parametri su presudni za tačnost segmentacije, jer direktno određuju opseg intenziteta piksela koji će biti interpretirani kao srčano tkivo u odnosu na okolne organe poput pluća ili kostiju.

Kako bi se osigurala robusnost sistema i omogućilo praćenje svake faze rekonstrukcije u realnom vremenu, razvijen je i integrisan namenski modul za logovanje. Klasa `Logger` automatski generiše tekstualni izveštaj sa vremenskim oznakama za svaki korak — od inicijalnog učitavanja volumena, preko složenih morfoloških operacija, pa sve do finalnog eksporta 3D mreže u STL format. Ovakav sistem ne samo da dokumentuje performanse *pipeline*-a, već služi i kao dijagnostički alat koji beleži kvantitativne metrike poput broja vokseli i zapremine srca, čime se potvrđuje integritet generisanog anatomskog modela.

U okviru razvoja sistema za 3D rekonstrukciju analizirana su dva različita pristupa konfiguraciji parametara segmentacije i obrade medicinskih slika. Prvi pristup podrazumeva korišćenje šireg opsega Hounsfield-ovih jedinica (od -1024 do 3071 HU), kao i višu gornju

granicu segmentacije (do 700 HU), čime se omogućava očuvanje kalcifikacija i finih vaskularnih struktura karakterističnih za koronarne arterije. Takođe, korišćen je manji prag za uklanjanje malih objekata, što dodatno doprinosi zadržavanju tankih anatomskih detalja. Nasuprot tome, drugi pristup koristi sužen HU opseg i nižu gornju granicu segmentacije (do 300 HU), uz veći prag za uklanjanje objekata, što dovodi do gubitka sitnijih struktura i pogodnije je za rekonstrukciju većih anatomskih celina, poput celokupnog srčanog tkiva. Na osnovu sprovedene analize zaključeno je da prvi pristup daje preciznije rezultate u kontekstu rekonstrukcije koronarnih arterija, jer omogućava bolju detekciju i očuvanje njihovih morfoloških karakteristika.

```
class Logger:
    """Klasa za logovanje sa timestamp-om"""

    def __init__(self, log_file):
        self.log_file = log_file
        self.start_time = time.time()

        with open(self.log_file, 'w', encoding='utf-8') as f:
            f.write("="*80 + "\n")
            f.write("PROGRAM ZA GENERISANJE 3D MODELA SRCA - LOG IZVEŠTAJ\n")
            f.write("="*80 + "\n")
            f.write(f"Vreme pokretanja: {time.strftime('%Y-%m-%d %H:%M:%S')}\n\n")

    def log(self, message, print_console=True):
        """Loguj poruku u fajl i opciono na konzolu"""
        timestamp = time.time() - self.start_time
        log_msg = f"[{timestamp:7.2f}s] {message}"

        if print_console:
            print(log_msg)

        with open(self.log_file, 'a', encoding='utf-8') as f:
            f.write(log_msg + "\n")

    def section(self, title):
        """Napravi sekciju u logu"""
        separator = "\n" + "="*80
        section_msg = f"{separator}\n{title}\n{separator}"
        self.log(section_msg)

    def format_number(self, num):
        """Formatiranje brojeva sa separatorima hiljada"""
        return f"{num:,}".replace(',', '.')
```

Slika 7-definisanje Logger klase(prvi deo)

```

def calculate_statistics(array):
    """Izračunaj osnovne statistike niza"""
    return {
        'min': np.min(array),
        'max': np.max(array),
        'mean': np.mean(array),
        'median': np.median(array),
        'std': np.std(array),
        'percentile_5': np.percentile(array, 5),
        'percentile_95': np.percentile(array, 95)
    }

def load_and_validate_data(input_dir, logger):

    logger.section("1. UČITAVANJE PODATAKA")

    if not os.path.exists(input_dir):
        logger.log(f"✗ GREŠKA: Direktorijum ne postoji: {input_dir}")
        raise FileNotFoundError(f"Direktorijum ne postoji: {input_dir}")

    png_files = sorted([f for f in os.listdir(input_dir) if f.lower().endswith('.png')])

    if not png_files:
        logger.log(f"✗ GREŠKA: Nisu pronađeni PNG fajlovi u: {input_dir}")
        raise FileNotFoundError("Nisu pronađeni PNG fajlovi!")

    logger.log(f"✓ Pronađeno {len(png_files)} PNG fajlova")
    logger.log(f"  Prvi fajl: {png_files[0]}")
    logger.log(f"  Poslednji fajl: {png_files[-1]}")

    first_slice = imageio.imread(os.path.join(input_dir, png_files[0]), mode='L')
    logger.log(f"  Dimenzije slajsa: {first_slice.shape[0]} x {first_slice.shape[1]}")
    logger.log(f"  Pixel opseg: [{first_slice.min()}, {first_slice.max()}]")

```

Slika 8 – definisanje Logger klase (drugi deo)

```

    logger.log("\n Učitavanje slajseva...")
    slices = []
    for i, f in enumerate(png_files):
        if (i+1) % 50 == 0:
            logger.log(f"  Učitano {i+1}/{len(png_files)} slajseva...", print_console=False)

            slice_img = imageio.imread(os.path.join(input_dir, f), mode='L')
            slices.append(slice_img)

    volume = np.stack(slices, axis=2)
    logger.log(f"\n✓ Volumen kreiran: shape = {volume.shape}")

    stats = calculate_statistics(volume)
    logger.log(f"  Grayscale statistike:")
    logger.log(f"    Min/Max: [{stats['min']:.1f}, {stats['max']:.1f}]")
    logger.log(f"    Mean ± Std: {stats['mean']:.1f} ± {stats['std']:.1f}")

    return volume

def convert_to_HU(volume, logger):
    """
    Konvertuje grayscale PNG vrednosti (0-255) u simulirane Hounsfield Units

    Mapiranje:
        0 (crno) → HU_MIN (npr. -500 za pluća)
        255 (belo) → HU_MAX (npr. 1000 za kost)
    """
    logger.section("2. KONVERZIJA U HOUNSFIELD UNITS (HU)")

    logger.log(f"  Parametri:")
    logger.log(f"    HU_MIN = {config.HU_MIN}")
    logger.log(f"    HU_MAX = {config.HU_MAX}")

    volume_normalized = volume.astype(float) / 255.0

```

Slika 9 – definisanje Logger klase(treci deo)

```
volume_HU = volume_normalized * (config.HU_MAX - config.HU_MIN) + config.HU_MIN

stats = calculate_statistics(volume_HU)
logger.log(f"\n ✓ HU konverzija završena:")
logger.log(f"    HU opseg: [{stats['min']:.1f}, {stats['max']:.1f}]")
logger.log(f"    Mean HU: {stats['mean']:.1f}")
logger.log(f"    Median HU: {stats['median']:.1f}")

return volume_HU

def resample_and_preprocess(volume_HU, logger):
    """
    Izotropno resamplovanje i Gaussian filtriranje
    """
    logger.section("3. RESAMPLING I PREPROCESSING")

    logger.log(f"    Original spacing: {config.SPACING_MM} mm")

    zoom_factors = config.SPACING_MM
    logger.log(f"    Zoom faktori: {zoom_factors}")

    logger.log(f"\n    Resamplovanje volumena...")
    volume_iso = zoom(volume_HU, zoom_factors, order=1)
    logger.log(f"    ✓ Shape nakon resamplovanja: {volume_iso.shape}")

    logger.log(f"\n    Gaussian smoothing (sigma={config.GAUSSIAN_SIGMA})...")
    volume_smooth = gaussian(volume_iso, sigma=config.GAUSSIAN_SIGMA, preserve_range=True)
    logger.log(f"    ✓ Smoothing završen")

    stats = calculate_statistics(volume_smooth)
    logger.log(f"\n    HU statistike nakon preprocessinga:")
    logger.log(f"    Mean: {stats['mean']:.1f}")
    logger.log(f"    Std: {stats['std']:.1f}")
    logger.log(f"    5th-95th percentile: [{stats['percentile_5']:.1f}, {stats['percentile_95']:.1f}]")
```

Slika 10 – definisanje Logger klase(cetvrti deo)

U odnosu na prvobitnu verziju loggera, koja je korišćena za obradu PNG slika srca, u drugoj verziji izvršene su značajne izmene kako bi se sistem prilagodio radu sa DICOM podacima i analizom koronarnih arterija. Najvažnija promena odnosi se na proširenje funkcionalnosti učitavanja podataka, gde logger sada prati validaciju DICOM fajlova, očitavanje medicinskih metapodataka (kao što su PixelSpacing, SliceThickness, RescaleSlope i RescaleIntercept), kao i automatsku konverziju u realne Hounsfield jedinice. Takođe, unapređena je kontrola grešaka kroz dodatne provere postojanja podataka i kompatibilnosti fajlova.

```

try:
    slope = float(ref.RescaleSlope)
    intercept = float(ref.RescaleIntercept)
    logger.log(f" ✓ RescaleSlope={slope}, RescaleIntercept={intercept}")
    hu_calibration = (slope, intercept)
except Exception:
    logger.log(" ⚠ RescaleSlope/Intercept nisu nađeni, koristiće se linearna normalizacija")
    hu_calibration = None
logger.log(f"\n Slaganje {len(valid_slices)} slajseva u 3D volumen...")
slices_arr = []
for ds in valid_slices:
    arr = ds.pixel_array.astype(np.float32)
    if hu_calibration:
        slope, intercept = hu_calibration
        arr = arr * slope + intercept
    slices_arr.append(arr)
volume = np.stack(slices_arr, axis=2)
logger.log(f" ✓ Volumen kreiran: shape = {volume.shape}")

stats = calculate_statistics(volume)
logger.log(f" HU statistike (ili raw vrednosti):")
logger.log(f" Min/Max: [{stats['min']:.1f}, {stats['max']:.1f}]")
logger.log(f" Mean ± Std: {stats['mean']:.1f} ± {stats['std']:.1f}")

return volume, actual_spacing

```

Slika 11- Implementacija 3D rekonstrukcije volumena uz kalibraciju sirovih piksela u Hounsfieldove (HU) jedinice radi osiguranja fizičke tačnosti podataka

Dalje, u drugoj verziji logger detaljnije beleži parametre segmentacije specifične za koronarne arterije, uključujući širi HU opseg (npr. 40–700 HU) radi obuhvatanja kalcifikacija, kao i prilagođene morfološke operacije za rad sa tanjim strukturama.

3.3. Učitavanje podataka i priprema volumena

Proces rekonstrukcije započinje automatizovanim učitavanjem serije dvodimenzionalnih snimaka koji formiraju trodimenzionalni skalarni skup podataka. Sistem je dizajniran da prepozna i validira PNG datoteke konvertovane iz DICOM standarda, obezbeđujući da su svi slojevi (slajsovi) konzistentnih dimenzija. Nakon učitavanja, pojedinačni snimci se slažu u 3D matricu (volumen) koristeći NumPy biblioteku, što omogućava efikasnu numeričku obradu u narednim fazama.

```

def load_and_validate_data(input_dir, logger):

    logger.section("1. UČITAVANJE PODATAKA")

    if not os.path.exists(input_dir):
        logger.log(f"✗ GREŠKA: Direktorijum ne postoji: {input_dir}")
        raise FileNotFoundError(f"Direktorijum ne postoji: {input_dir}")

    png_files = sorted([f for f in os.listdir(input_dir) if f.lower().endswith('.png')])

    if not png_files:
        logger.log(f"✗ GREŠKA: Nisu pronadeni PNG fajlovi u: {input_dir}")
        raise FileNotFoundError("Nisu pronadeni PNG fajlovi!")

    logger.log(f"✓ Pronadeno {len(png_files)} PNG fajlova")
    logger.log(f"Prvi fajl: {png_files[0]}")
    logger.log(f"Poslednji fajl: {png_files[-1]}")

    first_slice = imageio.imread(os.path.join(input_dir, png_files[0]), mode='L')
    logger.log(f"Dimenzije slajsa: {first_slice.shape[0]} x {first_slice.shape[1]}")
    logger.log(f"Pixel opseg: [{first_slice.min()}, {first_slice.max()}]")

    logger.log("\n Učitavanje slajseva...")
    slices = []
    for i, f in enumerate(png_files):
        if (i+1) % 50 == 0:
            logger.log(f"Učitano {(i+1)/len(png_files)} slajseva...", print_console=False)

        slice_img = imageio.imread(os.path.join(input_dir, f), mode='L')
        slices.append(slice_img)

    volume = np.stack(slices, axis=2)
    logger.log(f"\n✓ Volumen kreiran: shape = {volume.shape}")

    stats = calculate_statistics(volume)
    logger.log(f"Grayscale statistike:")
    logger.log(f"Min/Max: [{stats['min']:.1f}, {stats['max']:.1f}]")
    logger.log(f"Mean ± Std: [{stats['mean']:.1f} ± {stats['std']:.1f}]")

    return volume

```

Slika 12 –Ucitavanje i validacija podataka

Ključni korak u ovoj fazi je konverzija intenziteta piksela (grayscale vrednosti 0–255) u standardizovane **Hounsfieldove jedinice (HU)**. Pošto PNG format ne čuva originalne radiološke gustine, sistem primenjuje linearno mapiranje na osnovu definisanih minimalnih i maksimalnih HU vrednosti (npr. od -500 za vazduh do 1000 za koštano tkivo). Ova simulacija omogućava da se dalja segmentacija vrši na osnovu realnih fizičkih svojstava tkiva, a ne samo na osnovu vizuelnog kontrasta slike.

```

all_files = os.listdir(input_dir)
dicom_files = sorted([
    f for f in all_files
    if f.lower().endswith('.dcm')
    or f.lower().endswith('.dicom')
    or (not '.' in f)
])

if not dicom_files:
    dicom_files = sorted([
        f for f in all_files
        if os.path.isfile(os.path.join(input_dir, f))
    ])

```

Slika 13- Implementacija algoritma za detekciju i selekciju DICOM datoteka iz ulaznog direktorijuma

Dok smo za PNG datoteke koristili jednostavan pristup učitavanja celokupnog sadržaja, za **DICOM datoteke** primenjen je specijalizovan algoritam za detekciju i selekciju. Ovaj proces koristi dvoslojnu logiku: primarna selekcija prepoznaje datoteke prema standardnim ekstenzijama (.dcm, .dicom) ili specifičnim formatima bez ekstenzije, dok se u slučaju odsustva takvih datoteka aktivira rezervni **fallback** mehanizam koji validira sve prisutne fajlove u direktorijumu. Ovakva implementacija osigurava visoku **interoperabilnost** softvera, jer je u medicinskom inženjeringu neophodno da sistem samostalno prepozna i klasifikuje heterogene strukture podataka, nezavisno od načina na koji su arhivirane na disku

```
def convert_to_HU(volume, logger):
    """
    Konvertuje grayscale PNG vrednosti (0-255) u simulirane Hounsfield Units

    Mapiranje:
    0 (crno) → HU_MIN (npr. -500 za pluća)
    255 (belo) → HU_MAX (npr. 1000 za kost)
    """
    logger.section("2. KONVERZIJA U HOUNSFIELD UNITS (HU)")

    logger.log(f" Parametri:")
    logger.log(f" HU_MIN = {config.HU_MIN}")
    logger.log(f" HU_MAX = {config.HU_MAX}")

    volume_normalized = volume.astype(float) / 255.0

    volume_HU = volume_normalized * (config.HU_MAX - config.HU_MIN) + config.HU_MIN

    stats = calculate_statistics(volume_HU)
    logger.log(f"\n✓ HU konverzija završena:")
    logger.log(f" HU opseg: [{stats['min']:.1f}, {stats['max']:.1f}]")
    logger.log(f" Mean HU: {stats['mean']:.1f}")
    logger.log(f" Median HU: {stats['median']:.1f}")

    return volume_HU
```

Slika 14- Konverzija u Hounsfield Units (HU)

Dodatno, vrši se izotropno resamplovanje volumena. Kako su CT snimci često nejednake rezolucije po osama (npr. veći razmak između slojeva nego unutar samog sloja), primenjuje se interpolacija (zoom faktor) kako bi se postigao ujednačen *spacing* u svim pravcima. Na kraju, radi uklanjanja šuma koji nastaje tokom snimanja, primenjuje se Gausov filter, čime se dobija "glatki" volumen pogodan za preciznu detekciju ivica srca. Jedina razlika između PNG algoritma i DICOM algoritma je ta što prosledjujem spacing a svaki pacijent/snimak može imati drugačije parametre rezolucije.

```
def resample_and_preprocess(volume_HU, logger):
    """
    Izotropno resamplovanje i Gaussian filtriranje
    """
    logger.section("3. RESAMPLING I PREPROCESSING")

    logger.log(f" Original spacing: {config.SPACING_MM} mm")

    zoom_factors = config.SPACING_MM
    logger.log(f" Zoom faktori: {zoom_factors}")

    logger.log(f"\n Resamplovanje volumena...")
    volume_iso = zoom(volume_HU, zoom_factors, order=1)
    logger.log(f" ✓ Shape nakon resamplovanja: {volume_iso.shape}")

    logger.log(f"\n Gaussian smoothing (sigma={config.GAUSSIAN_SIGMA})...")
    volume_smooth = gaussian(volume_iso, sigma=config.GAUSSIAN_SIGMA, preserve_range=True)
    logger.log(f" ✓ Smoothing završen")

    stats = calculate_statistics(volume_smooth)
    logger.log(f"\n HU statistike nakon preprocessinga:")
    logger.log(f" Mean: {stats['mean']:.1f}")
    logger.log(f" Std: {stats['std']:.1f}")
    logger.log(f" 5th-95th percentile: [{stats['percentile_5']:.1f}, {stats['percentile_95']:.1f}]")

    return volume_smooth
```

Slika 15- Resamplovanje i inicijalno filtriranje (Preprocessing)

3.4. Robusna HU normalizacija

Robusna HU normalizacija implementirana je u okviru funkcije `convert_to_HU`. S obzirom na to da ulazni podaci dolaze u vidu 8-bitnih PNG slika, proces normalizacije je neophodan kako bi se povratila fizička interpretacija intenziteta piksela. Sistem vrši linearno mapiranje originalnog dinamičkog opsega na standardizovanu **Hounsfieldovu skalu** koristeći parametre `HU_MIN` (-500) i `HU_MAX` (1000).

Prvi deo su definisane granice u `Config` klasi koje određuju kako će se vršiti mapiranje.

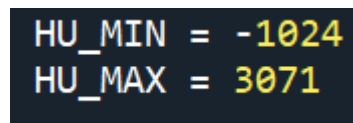
```
HU_MIN = -500
HU_MAX = 1000
```

Slika 16 - Parametri za normalizaciju (Konfiguracija)

Ovaj korak je kritičan za robusnost pipeline-a jer omogućava da se u kasnijim fazama segmentacije koriste fiksni radiološki pragovi za srčano tkivo (40–300 HU), nezavisno od varijacija u osvetljenosti ili kontrastu originalnih ulaznih slika. Validacija ovog procesa vrši se automatskim generisanjem histograma distribucije, čime se osigurava statistička ispravnost konverzije pre same 3D rekonstrukcije.

Nasuprot tome, direktna obrada DICOM podataka koristi izvornu Hounsfieldovu skalu u dijagnostičkom rasponu od -1024 do 3071 HU, čime se zadržava puna fizička dubina snimka

bez gubitaka usled kompresije. Dok se za detekciju srčanog tkiva primenjuje opšti prag (40–300 HU), za segmentaciju koronarnih arterija koristi se prošireni opseg (40–700 HU) koji omogućava pouzdano izdvajanje kontrastom ispunjenih vaskularnih struktura i kalcifikacija.



```
HU_MIN = -1024
HU_MAX = 3071
```

Slika 17-Prametri normalizacije(koronarna arterija)

3.5. Izotropno resamplovanje

U procesu medicinske obrade slika, CT snimci često imaju **neujednačen razmak između voxela** u različitim osama (X, Y, Z). Ovo znači da dimenzije piksela u ravni slike nisu jednake dimenziji između slojeva.

U ovom radu primenjuje se **izotropno resamplovanje**, kojim se postiže da svi voxeli imaju jednake dimenzije u sve tri dimenzije.

Korišćen je sledeći pristup:

- Ulazni spacing: **$0.8 \times 0.8 \times 1.0$ mm** ili $1.0 \times 1.0 \times 1.0$ mm za drugi slučaj
- Primenjena funkcija: `scipy.ndimage.zoom`
- Interpolacija: linearna (order=1)

Cilj ovog koraka je:

- eliminacija deformacija u Z-osi
- omogućavanje precizne 3D rekonstrukcije
- poboljšanje performansi algoritma marching cubes

Nakon resamplovanja, volumen postaje **geometrijski konzistentan**, što je ključno za dalju obradu.

3.6. Brzi informativni ispis (volumen, HU opseg, prag, maska)

U cilju kontrole kvaliteta podataka, implementiran je sistem za brzi informativni ispis koji omogućava uvid u osnovne karakteristike volumena i rezultate segmentacije. Ispis obuhvata dimenzije volumena, opseg intenziteta (min/max), srednju vrednost i standardnu devijaciju, HU opseg nakon konverzije, kao i broj voksela u segmentiranoj maski.

Ovaj korak omogućava brzo otkrivanje grešaka — mali broj vokselu ukazuje na loš izbor praga, dok preveliki broj znači da su uključeni neželjeni regioni.

Ključni delovi koda koji pealizuju ovaj ispis su:

```
def print_summary(volume, volume_HU, mask):
    print("\nINFORMACIJE O CT PODACIMA")
    print(f"• Volumen (Z, Y, X): {volume.shape}")
    print(f"• Spacing (mm): {config.SPACING_MM}")

    voxel_size = config.SPACING_MM[0] * config.SPACING_MM[1] *
config.SPACING_MM[2]
    print(f"• Veličina voxela (mm³): {voxel_size:.2f}")

    print(f"• HU opseg: {volume_HU.min():.1f} → {volume_HU.max():.1f}")

    voxels_mask = int(mask.sum())
    frac = voxels_mask / mask.size * 100
    print(f"• Vokselu u maski: {voxels_mask} ({frac:.2f}% volumena)|")
```

Slika 18: Sažeti ispis ključnih metapodataka

3.7. Multiplanarna vizuelizacija i validacija segmentacije

U okviru vizuelne validacije podataka, vrši se multiplanarni prikaz središnjih preseka resamplovanog volumena kroz tri osnovne ortogonalne ravni: aksijalnu, sagitalnu i koronalnu. Gornji red prikaza fokusiran je na sirove podatke u intenzitetskom opsegu od -200 do 300 HU, što je optimalno za vizuelizaciju mekih tkiva i proveru anatomske koherentnosti srca unutar vidnog polja. Donji red direktno nadovezuje rezultate segmentacije na originalne snimke korišćenjem polutransparentne crvene maske, čime se omogućava neposredna provera tačnosti algoritma i preciznosti izdvojenih kontura u odnosu na okolne strukture. Ovakav sveobuhvatan pregled služi kao ključni kontrolni korak za ranu detekciju eventualnog šuma ili diskontinuiteta, potvrđujući da su podaci spremni za finalnu fazu 3D rekonstrukcije. U oba primera je koriscen sustinski slican kod.

```

z_mid = volume.shape[2] // 2
y_mid = volume.shape[1] // 2
x_mid = volume.shape[0] // 2

fig, axes = plt.subplots(2, 3, figsize=(18, 12))

vmin, vmax = -200, 300

axes[0, 0].imshow(volume[:, :, z_mid].T, cmap='gray', vmin=vmin, vmax=vmax, origin='lower')
axes[0, 0].set_title(f"Axial presek (Z={z_mid})", fontsize=12, fontweight='bold')
axes[0, 0].axis('off')

axes[0, 1].imshow(volume[x_mid, :, :].T, cmap='gray', vmin=vmin, vmax=vmax, origin='lower')
axes[0, 1].set_title(f"Sagittal presek (X={x_mid})", fontsize=12, fontweight='bold')
axes[0, 1].axis('off')

axes[0, 2].imshow(volume[:, y_mid, :].T, cmap='gray', vmin=vmin, vmax=vmax, origin='lower')
axes[0, 2].set_title(f"Coronal presek (Y={y_mid})", fontsize=12, fontweight='bold')
axes[0, 2].axis('off')

axes[1, 0].imshow(volume[:, :, z_mid].T, cmap='gray', vmin=vmin, vmax=vmax, origin='lower')
axes[1, 0].imshow(mask[:, :, z_mid].T, cmap='Reds', alpha=0.5, origin='lower')
axes[1, 0].set_title("Axial + Segmentacija", fontsize=12, fontweight='bold')
axes[1, 0].axis('off')

axes[1, 1].imshow(volume[x_mid, :, :].T, cmap='gray', vmin=vmin, vmax=vmax, origin='lower')
axes[1, 1].imshow(mask[x_mid, :, :].T, cmap='Reds', alpha=0.5, origin='lower')
axes[1, 1].set_title("Sagittal + Segmentacija", fontsize=12, fontweight='bold')
axes[1, 1].axis('off')

axes[1, 2].imshow(volume[:, y_mid, :].T, cmap='gray', vmin=vmin, vmax=vmax, origin='lower')
axes[1, 2].imshow(mask[:, y_mid, :].T, cmap='Reds', alpha=0.5, origin='lower')
axes[1, 2].set_title("Coronal + Segmentacija", fontsize=12, fontweight='bold')
axes[1, 2].axis('off')

```

Slika 19- Uporedni pregled središnjih aksijalnih, sagitalnih i koronalnih preseka originalnog volumena (gornji red) i njihovo preklapanje sa dobijenom maskom segmentacije (donji red) radi validacije prostorne tačnosti modela

3.8. Analiza osetljivosti HU pragova na kvalitet segmentacije

Kako bi se odredili optimalni parametri za izolaciju srca, sprovedena je studija uticaja različitih Hounsfield-ovih pragova na finalnu masku. Poređenjem tri skupa vrednosti — nižeg (30-200 HU), standardnog (40-300 HU) i višeg praga (50-400 HU) — vizuelno je analizirana preciznost obuhvatanja vokseli i eliminacija okolnog šuma. Prikaz broja detektovanih vokseli za svaku opciju omogućava kvantitativnu procenu robusnosti segmentacije, osiguravajući da izabrani parametri najbolje reprezentuju anatomiju srca bez uključivanja neželjenih struktura.

```

def plot_threshold_study(volume, output_dir, logger):
    """
    Vizuelno poređenje različitih HU pragova
    """
    logger.log("\n [Vizualizacija] Studija uticaja pragova...")

    z_mid = volume.shape[2] // 2
    slice_img = volume[:, :, z_mid].T

    thresholds = [
        (30, 200, "Niži prag (30-200 HU)"),
        (40, 300, "Standardni (40-300 HU)"),
        (50, 400, "Viši prag (50-400 HU)")
    ]

    fig, axes = plt.subplots(2, 3, figsize=(18, 12))

    for idx, (lower, upper, title) in enumerate(thresholds):

        axes[0, idx].imshow(slice_img, cmap='gray', vmin=-200, vmax=300, origin='Lower')
        axes[0, idx].set_title(f"{title}\nOriginalni presek", fontsize=11)
        axes[0, idx].axis('off')

        mask_test = (volume[:, :, z_mid] > lower) & (volume[:, :, z_mid] < upper)

        axes[1, idx].imshow(slice_img, cmap='gray', vmin=-200, vmax=300, origin='Lower')
        axes[1, idx].imshow(mask_test.T, cmap='Reds', alpha=0.5, origin='Lower')

        n_voxels = np.sum(mask_test)
        axes[1, idx].set_title(
            f"Segmentacija\n{format_number(n_voxels)} voksela",
            fontsize=11
        )
        axes[1, idx].axis('off')

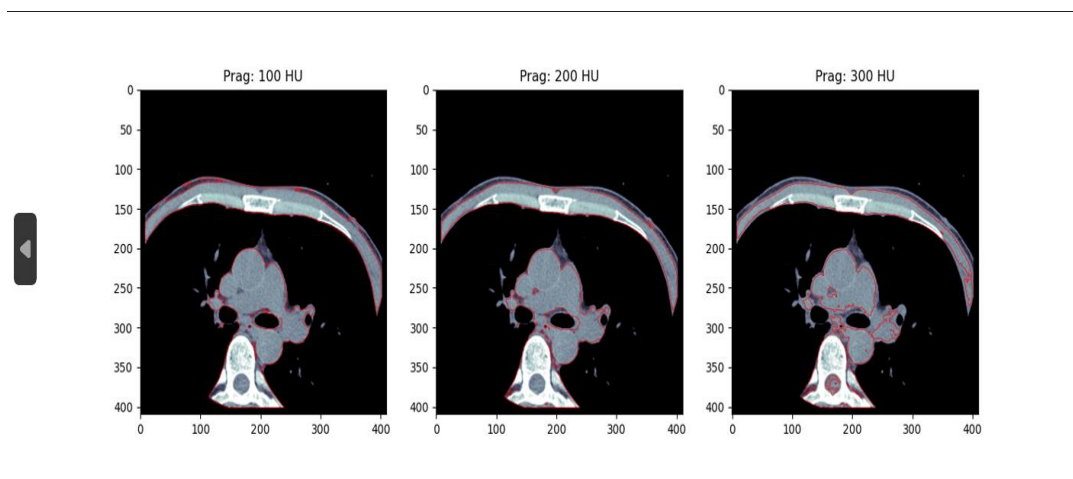
    plt.suptitle(
        "Studija Uticaja HU Pragova na Segmentaciju",
        fontsize=16,
        fontweight='bold'
    )
    plt.tight_layout()

    output_path = os.path.join(output_dir, "03_threshold_study.png")
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()

    logger.log(f"    ✓ Studija pragova sačuvana: {output_path}")

```

Slika 20- Upporednu analizu tri različita HU praga kroz vizuelizaciju maske i broj detektovanih voksela,



Slika 21-Studija uticaja pragova

Kako bi se osigurala optimalna segmentacija koronarnih arterija, sprovedena je studija osetljivosti na promenu Hounsfieldovih pragova. Analizirani su različiti opsezi: od standardnog (40–700 HU) do specijalizovanog opsega za detekciju kalcifikacija (130–700 HU). Poređenjem vizuelnih rezultata i broja izdvojenih vokselâ, definisan je optimalan prag koji omogućava balans između uklanjanja šuma i očuvanja integriteta vaskularnih struktura. Ova metodologija omogućava empirijsko dokazivanje izbora parametara, čime se proces segmentacije čini naučno fundiranim i ponovljivim. Na sledecoj slici je to detaljno prikazano na tacaan nacin:

```
def plot_threshold_study(volume, output_dir, logger):
    logger.log("\n [Viz] Studija HU pragova...")
    z_mid = volume.shape[2] // 2
    slice_img = volume[:, :, z_mid].T
    thresholds = [
        (30, 400, "Niži prag (30-400 HU)"),
        (40, 700, "Standardni (40-700 HU)"),
        (130, 700, "Kalcifikacije (130-700 HU)"),
    ]
    fig, axes = plt.subplots(2, 3, figsize=(18, 12))
    for idx, (lower, upper, title) in enumerate(thresholds):
        axes[0, idx].imshow(slice_img, cmap='gray', vmin=-200, vmax=500, origin='Lower')
        axes[0, idx].set_title(title, fontsize=11)
        axes[0, idx].axis('off')
        mask_test = (volume[:, :, z_mid] > lower) & (volume[:, :, z_mid] < upper)
        axes[1, idx].imshow(mask_test.T, cmap='Reds', alpha=0.5, origin='Lower')
        axes[1, idx].set_title(f"{format_number(np.sum(mask_test))} vokselâ", fontsize=11)
        axes[1, idx].axis('off')
    plt.suptitle("Studija HU Pragova – Koronarne Arterije", fontsize=16, fontweight='bold')
    plt.tight_layout()
    out = os.path.join(output_dir, "03_threshold_study.png")
    plt.savefig(out, dpi=300, bbox_inches='tight')
    plt.close()
    logger.log(f" ✓ {out}")
```

Slika 22- Implementacija algoritma za empirijsku studiju osetljivosti Hounsfieldovih pragova koronarna arterija

3.9. HU histogram i vizuelna analiza segmentacionih pragova

Kragujevac, 2026.

Funkcija `plot_hu_histogram` služi za vizuelnu analizu raspodele Hounsfieldovih jedinica (HU) u CT volumenu, što je ključni korak u obradi medicinskih snimaka grudnog koša. Na ulazu prima 3D niz HU vrednosti (`volume_HU`), izlazni direktorijum za čuvanje rezultata i logger za praćenje procesa. Prvo se svi vokseli volumena „spljošte“ u 1D niz kako bi se dobila ukupna statistička raspodela tkiva u snimku. Zatim se crta histogram sa 300 binova u opsegu od -500 do 500 HU, što omogućava jasnu diferencijaciju različitih tipova tkiva, poput vazduha u plućima, mekih tkiva i struktura viših gustina. Posebna vrednost funkcije je vizuelno označavanje segmentacionih pragova za srce: donja i gornja granica (`HEART_HU_LOWER` i `HEART_HU_UPPER`) prikazane su vertikalnim linijama, dok je opseg između njih dodatno istaknut zelenom providnom zonom, čime se naglašava region od interesa za srčanu segmentaciju. Graf je dodatno obogaćen naslovom, legendom, mrežom i anotacijama koje označavaju približne regije pluća i mekih tkiva, što olakšava interpretaciju. Na kraju se histogram optimizovano čuva kao slika visoke rezolucije u izlaznom direktorijumu, čime se omogućava dalja upotreba u izveštaju ili analizi rezultata segmentacije.

```

def plot_hu_histogram(volume_HU, output_dir, logger):
    """
    Histogram HU vrednosti sa označenim segmentacijskim pragovima
    """
    logger.log("\n [Vizualizacija] Kreiranje HU histograma...")

    fig, ax = plt.subplots(figsize=(14, 6))
    counts, bins, patches = ax.hist(
        volume_HU.flatten(),
        bins=300,
        range=(-500, 500),
        color='steelblue',
        alpha=0.7,
        edgecolor='black',
        linewidth=0.5
    )
    ax.axvline(
        config.HEART_HU_LOWER,
        color='red',
        linestyle='--',
        linewidth=2.5,
        label=f'Donja granica srca ({config.HEART_HU_LOWER} HU)'
    )
    ax.axvline(
        config.HEART_HU_UPPER,
        color='orange',
        linestyle='--',
        linewidth=2.5,
        label=f'Gornja granica srca ({config.HEART_HU_UPPER} HU)'
    )
    ax.axvspan(
        config.HEART_HU_LOWER,
        config.HEART_HU_UPPER,
        alpha=0.2,
        color='green',
        label='Srčani region'
    )
    ax.set_xlabel("Hounsfield Units (HU)", fontsize=13, fontweight='bold')
    ax.set_ylabel("Frekvencija (broj vokselâ)", fontsize=13, fontweight='bold')
    ax.set_title(
        "Histogram HU Vrednosti - CT Snimak Grudnog Koša",
        fontsize=15,
        fontweight='bold',
        pad=20
    )
    ax.legend(fontsize=11, loc='upper right', framealpha=0.9)
    ax.grid(True, alpha=0.3, linestyle=':', linewidth=0.8)
    ax.set_xlim(-500, 500)

    ax.text(-400, ax.get_ylim()[1]*0.9, 'Pluća', fontsize=10, color='darkblue', style='italic')
    ax.text(150, ax.get_ylim()[1]*0.7, 'Srce/Trkivo', fontsize=10, color='darkred', style='italic')

    plt.tight_layout()
    output_path = os.path.join(output_dir, "01_hu_histogram.png")
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    plt.close()

    logger.log(f"    ✓ Histogram sačuvan: {output_path}")

```

Slika 23- HU histogram sa označenim segmentacionim pragovima

Sličan kod sa malim razlikama korišćen je za vizuelizaciju histograma koronarnih arterija, pri čemu je primarna razlika u **dijagnostičkom fokusu**. Dok opšti histogram prikazuje distribuciju celog volumena grudnog koša, histogram za koronarne arterije se fokusira na uži, visokodenzitetni segment, što je ključno za izdvajanje vaskularnih struktura.

3.10. Napredna segmentacija srčanog regiona u CT volumenu

Funkcija `segment_heart` implementira napredni pipeline za segmentaciju srčanog tkiva iz CT volumena zasnovanog na Hounsfieldovim jedinicama (HU). Proces započinje pragovanjem intenziteta u opsegu definisanom parametrima `HEART_HU_LOWER` i `HEART_HU_UPPER`, čime se izdvaja inicijalni binarni maskirani region koji potencijalno sadrži srčano tkivo.

Nakon inicijalnog izdvajanja, primenjuje se morfološka operacija **binary closing** kako bi se popunile manje šupljine i poboljšala kontinuitet strukture. Zatim se vrši popunjavanje unutrašnjih šupljina (hole filling), čime se obezbeđuje kompaktnija i anatomski realnija forma segmentiranog regiona.

U sledećem koraku koristi se analiza povezanih komponenti (connected components analysis), gde se identifikuju sve odvojene strukture u volumenu, a zatim se zadržava najveća komponenta kao najverovatniji kandidat za srce. Ovaj pristup eliminiše neželjene strukture slične gustine koje mogu biti prisutne u inicijalnom thresholdingu.

Na kraju se primenjuje filtriranje malih objekata kako bi se uklonili šum i preostale sitne artefaktne regije. Rezultat predstavlja binarnu masku srčanog tkiva, spremnu za dalju 3D vizualizaciju ili analizu. Ovaj višestepeni pristup omogućava robusnu segmentaciju zasnovanu na kombinaciji intenzitetskih i topoloških karakteristika volumena.

```
def segment_heart(volume, logger):
    """
    Napredna segmentacija srčanog tkiva

    Pipeline:
    1. HU thresholding (40-300 HU)
    2. Binary closing (popunjavanje malih rupa)
    3. Hole filling (popunjavanje velikih šupljina)
    4. Connected components analysis (izdvajanje srca)
    5. Small objects removal
    """
    logger.section("4. SEGMENTACIJA SRCA")

    logger.log(f" Parametri segmentacije:")
    logger.log(f"   HU donja granica: {config.HEART_HU_LOWER}")
    logger.log(f"   HU gornja granica: {config.HEART_HU_UPPER}")

    logger.log(f"\n [Korak 1] HU pragovanje...")
    binary_volume = (volume > config.HEART_HU_LOWER) & (volume < config.HEART_HU_UPPER)
    n_voxels_after_threshold = np.sum(binary_volume)
    logger.log(f"   Vokseli nakon pragovanja: {format_number(n_voxels_after_threshold)}")

    logger.log(f"\n [Korak 2] Binary closing (radius={config.CLOSING_RADIUS})...")
    struct_elem = ball(config.CLOSING_RADIUS)
    binary_volume = binary_closing(binary_volume, struct_elem)
    logger.log(f"   ✓ Closing završen")

    logger.log(f"\n [Korak 3] Popunjavanje šupljina...")
    binary_volume = binary_fill_holes(binary_volume)
    logger.log(f"   ✓ Holes filled")

    logger.log(f"\n [Korak 4] Connected components analiza...")
    labels = measure.label(binary_volume)
    n_components = labels.max()
    logger.log(f"   Pronađeno komponenti: {n_components}")
```

Slika 24-Napredna segmentacija srčanog tkiva(prvi deo)

```

if n_components == 0:
    logger.log(f"      ✗ UPOZORENJE: Nijedna komponenta nije detektovana!")
    logger.log(f"      Proveri HU parametre segmentacije.")
    return binary_volume

component_sizes = np.bincount(labels.flat)[1:]
largest_component_idx = np.argmax(component_sizes) + 1
largest_size = component_sizes[largest_component_idx - 1]

logger.log(f"      Najveća komponenta:")
logger.log(f"      Label: {largest_component_idx}")
logger.log(f"      Vokseli: {format_number(largest_size)}")
logger.log(f"      % od ukupnog volumena: {100*largest_size/binary_volume.size:.2f}%")

binary_volume = (labels == largest_component_idx)

logger.log(f"\n [Korak 5] Uklanjanje malih objekata (min_size={config.MIN_OBJECT_SIZE})...")
binary_volume_clean = remove_small_objects(binary_volume, min_size=config.MIN_OBJECT_SIZE)
removed_voxels = np.sum(binary_volume) - np.sum(binary_volume_clean)

if removed_voxels > 0:
    logger.log(f"      Uklonjeno {format_number(removed_voxels)} voksela")
else:
    logger.log(f"      Nema malih objekata za uklanjanje")

final_voxel_count = np.sum(binary_volume_clean)
logger.log(f"\n ✓ Segmentacija završena:")
logger.log(f"      Finalni broj voksela: {format_number(final_voxel_count)}")

return binary_volume_clean

```

Slika 25-Napredna segmentacija srčanog tkiva(drugi deo)

```

2.85s]   Parametri segmentacije:
2.85s]     HU donja granica: 40
2.85s]     HU gornja granica: 300
2.85s]
[Korak 1] HU pragovanje...
2.88s]     Voksela nakon pragovanja: 435.506
2.88s]
[Korak 2] Binary closing (radius=2)...
3.41s]     ✓ Closing završen
3.41s]
[Korak 3] Popunjavanje šupljina...
3.85s]     ✓ Holes filled
3.85s]
[Korak 4] Connected components analiza...
3.98s]     Pronađeno komponenti: 92
4.03s]     Najveća komponenta:
4.03s]       Label: 33
4.03s]       Vokseli: 315.985
4.03s]       % od ukupnog volumena: 3.76%
4.04s]
[Korak 5] Uklanjanje malih objekata (min_size=5000)...
4.22s]     Uklonjeno 1.246 voksela
4.22s]
✓ Segmentacija završena:
4.22s]     Finalni broj voksela: 314.739

```

Slika 26 – Rezultati prethodnih komandi

Iako je osnovna logika segmentacionog *pipeline*-a identična za oba slučaja, primenjena je adaptacija parametara za specifičnu obradu koronarnih arterija. Dok `segment_heart` koristi standardne pragove za srčano tkivo, modifikovana verzija algoritma primenjuje prošireni HU opseg (40–700 HU), čime se omogućava visoka selektivnost pri izdvajanju kontrastom ispunjenih vaskularnih struktura. Ovakva fleksibilnost algoritma potvrđuje njegovu modularnost i primenljivost na različite anatomske strukture unutar istog radiološkog sistema.

```
[ 4.05s] Parametri segmentacije:
[ 4.05s] HU donja granica: 40
[ 4.05s] HU gornja granica: 700
[ 4.05s]
[Korak 1] HU pragovanje...
[ 4.05s] Voksela nakon pragovanja: 1.302.897
[ 4.07s]
[Korak 2] Binary closing (radius=2)...
[ 4.32s] ✓ Closing završen
[ 4.32s]
[Korak 3] Popunjavanje šupljina...
[ 4.45s] ✓ Holes filled
[ 4.45s]
[Korak 4] Connected components analiza...
[ 4.50s] Pronađeno komponenti: 120
[ 4.53s] Najveća komponenta: 1.282.322 voksela (36.00% volumena)
[ 4.53s]
[Korak 5] Uklanjanje malih objekata (min_size=2000)...
[ 4.61s] Uklonjeno 173 voksela
[ 4.61s]
✓ Segmentacija završena: 1.282.149 voksela
[ 4.61s]
```

Slika 27-rezultati segmentacije koronarne arterije

3.11. Kvantitativna analiza srčanog volumena

Funkcija `calculate_volumetrics` vrši kvantitativnu analizu segmentiranog srčanog tkiva na osnovu binarne maske i informacija o prostornoj rezoluciji CT snimka (`spacing`). Osnovni cilj ove faze je određivanje volumena srca, kao i dodatnih geometrijskih karakteristika koje opisuju njegovu prostornu distribuciju.

Prvo se računa zapremina jednog voksela na osnovu fizičkih dimenzija u milimetrima, nakon čega se ukupan broj voksela u segmentiranoj maski množi sa tom vrednošću kako bi se dobio ukupan volumen u mm^3 , koji se zatim konvertuje u cm^3 radi lakše interpretacije u kliničkom kontekstu. Dobijeni volumen se upoređuje sa referentnim opsegom (200–250 cm^3), čime se omogućava osnovna provera da li rezultat odgovara očekivanim fiziološkim vrednostima.

```

def calculate_volumetrics(binary_mask, spacing, logger):
    """
    Izračunava volumen i površinu srca
    """
    logger.section("5. KVANTITATIVNA ANALIZA")

    voxel_volume_mm3 = spacing[0] * spacing[1] * spacing[2]
    total_voxels = np.sum(binary_mask)
    volume_mm3 = total_voxels * voxel_volume_mm3
    volume_cm3 = volume_mm3 / 1000.0

    logger.log(f"  Volumen:")
    logger.log(f"    Broj voksela: {format_number(total_voxels)}")
    logger.log(f"    Voxel dimenzija: {voxel_volume_mm3:.4f} mm³")
    logger.log(f"    Ukupan volumen: {volume_mm3:.1f} mm³ = {volume_cm3:.2f} cm³")

    if 400 <= volume_cm3 <= 800:
        logger.log(f"      ✓ Volumen u normalnom opsegu (400-800 cm³)")
    elif volume_cm3 < 400:
        logger.log(f"      ⚠ UPOZORENJE: Volumen manji od očekivanog (<400 cm³)")
    else:
        logger.log(f"      ⚠ UPOZORENJE: Volumen veći od očekivanog (>800 cm³)")

    coords = np.argwhere(binary_mask)
    bbox_min = coords.min(axis=0)
    bbox_max = coords.max(axis=0)
    bbox_size = (bbox_max - bbox_min + 1) * np.array(spacing)

    logger.log(f"\n  Bounding Box dimenzije (mm):")
    logger.log(f"    X (levo-desno): {bbox_size[0]:.1f} mm")
    logger.log(f"    Y (prednje-zadnje): {bbox_size[1]:.1f} mm")
    logger.log(f"    Z (gore-dole): {bbox_size[2]:.1f} mm")

    centroid = coords.mean(axis=0) * np.array(spacing)
    logger.log(f"\n  Centroid (mm):")
    logger.log(f"    X: {centroid[0]:.1f}, Y: {centroid[1]:.1f}, Z: {centroid[2]:.1f}")

    return {
        'volume_cm3': volume_cm3,
        'volume_mm3': volume_mm3,
        'bbox_size_mm': bbox_size,
        'centroid_mm': centroid,
        'n_voxels': total_voxels
    }

```

Slika 28-Kvalitativna analiza srca(volumen i površina)

Pored volumena, funkcija određuje i prostorni obuhvat srca kroz izračunavanje bounding box-a, koji predstavlja minimalni pravougaoni region koji sadrži sve voksele segmentiranog tkiva. Ovaj parametar daje uvid u dimenzije srca u sve tri prostorne ose (levo-desno, prednje-zadnje i gore-dole).

Dodatno se računa i centroid segmentiranog regiona, odnosno geometrijski centar mase srca, koji predstavlja prosečnu prostornu poziciju tkiva u 3D prostoru. Na kraju, funkcija vraća skup ključnih kvantitativnih parametara koji se mogu koristiti za dalje medicinske analize, vizualizacije ili poređenja između različitih pacijenata ili snimaka.

```

[ 4.23s] Volumen:
[ 4.23s] Broj vokselâ: 314.739
[ 4.23s] Voxel dimenzija: 0.6400 mm³
[ 4.23s] Ukupan volumen: 201433.0 mm³ = 201.43 cm³
[ 4.25s] ⚠ UPOZORENJE: Volumen manji od oĉekivanog (<400 cm³)
[ 4.28s]
[ 4.28s] Bounding Box dimenzije (mm):
[ 4.28s] X (levo-desno): 192.8 mm
[ 4.28s] Y (prednje-zadnje): 161.6 mm
[ 4.28s] Z (gore-dole): 50.0 mm
[ 4.28s]
[ 4.28s] Centroid (mm):
[ 4.28s] X: 235.5, Y: 158.2, Z: 24.6
[ 4.28s]

```

Slika 29-Rezultati analize PNG fajlovi

Radi postizanja visoke pouzdanosti prikaza, algoritam je optimizovan za **sveobuhvatnu segmentaciju srĉanog volumena(kod DICOM fajlova)**. Primenom Hounsfieldovog praga u opsegu od 40 do 700 HU, sistem uspešno izoluje celokupnu srĉanu strukturu, ukljuĉujući miokard i koronarno vaskularno stablo. Ovakav pristup omogućava vizuelizaciju anatomskih odnosa izmeĊu srĉanih komora i arterija unutar jedinstvenog 3D modela, što pruža kompletniji dijagnostiĉki uvid nego izdvajanje arterija kao zasebnih, izolovanih struktura.

```

[ 4.62s] Volumen: 1282149.0 mm³ = 1282.15 cm³
[ 4.65s] Bounding Box (mm): X=173.0 Y=173.0 Z=119.0
[ 4.65s] Centroid (mm): 78.0, 67.7, 56.3
[ 4.65s]

```

Slika 30-Rezultati analize DICOM fajlova

Kvantitativna analiza, koja je rezultovala volumenom od približno **1282 cm³**, direktno potvrĊuje da algoritam uspešno obuhvata celokupan srĉani volumen. Ovakav rezultat predstavlja ĉvrstu osnovu za dalju 3D rekonstrukciju (*Marching Cubes*), obezbeĊujući da anatomski model srca poseduje sve bitne komponente neophodne za njegovu vizuelizaciju i analizu.

3.12. Rekonstrukcija i obrada 3D modela srĉanog tkiva

Ovaj deo implementacije se odnosi na rekonstrukciju i vizualizaciju srĉanog tkiva u obliku 3D površinskog modela na osnovu binarne segmentacione maske. Proces zapoĉinje primenom **Marching Cubes algoritma**, koji iz volumetrijskih podataka ekstrahuje površinsku mrežu (mesh) definisanjem izopovršine na zadanom nivou praga. Kao rezultat dobijaju se skupovi temena (vertices), trouglova (faces) i normala, koji zajedno formiraju geometrijski prikaz srca u trodimenzionalnom prostoru.

```

def generate_3d_mesh(binary_mask, logger):
    """
    Marching cubes algoritam za generisanje 3D mreže
    """
    logger.section("6. GENERISANJE 3D MREŽE")

    logger.log(f" Pokretanje Marching Cubes algoritma...")
    logger.log(f" Level threshold: {config.MARCHING_CUBES_LEVEL}")

    verts, faces, normals, values = measure.marching_cubes(
        binary_mask,
        level=config.MARCHING_CUBES_LEVEL,
        spacing=(1.0, 1.0, 1.0)
    )

    logger.log(f"\n✓ Mreža generisana:")
    logger.log(f" Vertices (tačke): {format_number(len(verts))}")
    logger.log(f" Faces (trouglovi): {format_number(len(faces))}")
    logger.log(f" Normals: {format_number(len(normals))}")

    faces_pv = np.hstack([np.full((faces.shape[0], 1), 3), faces]).astype(np.int32)
    mesh = pv.PolyData(verts, faces_pv)

    mesh_stats = {
        'n_vertices': len(verts),
        'n_faces': len(faces),
        'surface_area': mesh.area,
        'bounds': mesh.bounds
    }

    logger.log(f"\n Mesh statistike:")
    logger.log(f" Površina: {mesh_stats['surface_area']:.2f} mm²")
    logger.log(f" Bounds (X): [{mesh_stats['bounds'][0]:.1f}, {mesh_stats['bounds'][1]:.1f}]")
    logger.log(f" Bounds (Y): [{mesh_stats['bounds'][2]:.1f}, {mesh_stats['bounds'][3]:.1f}]")
    logger.log(f" Bounds (Z): [{mesh_stats['bounds'][4]:.1f}, {mesh_stats['bounds'][5]:.1f}]")

    return mesh, mesh_stats

```

Slika 31- Marching Cubes algoritam

Nakon generisanja mreže, ona se konvertuje u odgovarajući format kompatibilan sa PyVista strukturom, što omogućava dalje geometrijske analize i vizualizaciju. Izračunavaju se ključne mesh statistike, uključujući broj temena i poligona, ukupnu površinu modela i njegove prostorne granice (bounding box), što omogućava kvantitativnu procenu dobijene 3D rekonstrukcije.

```

[ 4.28s] Pokretanje Marching Cubes algoritma...
[ 4.28s] Level threshold: 0.5
[ 4.66s]
✓ [ 4.66s] Mreža generisana:
[ 4.66s] Vertices (tačke): 276.574
[ 4.66s] Faces (trouglovi): 556.132
[ 4.66s] Normals: 276.574
[ 4.84s]
[ 4.84s] Mesh statistike:
[ 4.84s] Površina: 206758.70 mm²
[ 4.84s] Bounds (X): [168.5, 409.0]
[ 4.84s] Bounds (Y): [91.5, 293.5]
[ 4.84s] Bounds (Z): [0.0, 49.0]
[ 4.85s]

```

Slika 32-Rezultati izvršavanja Meshing Cubes algoritma

U nastavku se primenjuje Laplacian smoothing algoritam, čiji je cilj uklanjanje nepravilnosti i šuma sa površine modela, čime se dobija anatomski realističniji i vizuelno glađi prikaz srca. Smoothing se kontroliše brojem iteracija i faktorom relaksacije, koji određuju stepen izravnavanja površine. Nakon obrade, vrši se poređenje originalnog i filtriranog modela kroz broj tačaka, poligona i ukupnu površinu, čime se procenjuje uticaj filtriranja na geometriju modela.

```
def smooth_mesh(mesh, logger):
    """
    Laplacian smoothing za glatku anatomsku površinu
    """
    logger.section("7. MESH SMOOTHING")

    logger.log(f"  Parametri smoothing-a:")
    logger.log(f"    Iteracije: {config.SMOOTHING_ITERATIONS}")
    logger.log(f"    Relaxation faktor: {config.SMOOTHING_RELAXATION}")

    logger.log(f"\n  Pokretanje smoothing-a...")
    smoothed_mesh = mesh.smooth(
        n_iter=config.SMOOTHING_ITERATIONS,
        relaxation_factor=config.SMOOTHING_RELAXATION
    )

    logger.log(f"  ✓ Smoothing završen")

    logger.log(f"\n  Poređenje pre/posle smoothing-a:")
    logger.log(f"    Vertices: {mesh.n_points} → {smoothed_mesh.n_points}")
    logger.log(f"    Faces: {mesh.n_cells} → {smoothed_mesh.n_cells}")
    logger.log(f"    Površina: {mesh.area:.2f} → {smoothed_mesh.area:.2f} mm²")

    return smoothed_mesh
```

Slika 33- Laplacian smoothing algoritam

```
[ 4.85s] Parametri smoothing-a:
[ 4.85s] Iteracije: 500
[ 4.85s] Relaxation faktor: 0.05
[ 4.85s]
Pokretanje smoothing-a...
[ 18.47s] ✓ Smoothing završen
[ 18.47s]
Poređenje pre/posle smoothing-a:
[ 18.47s] Vertices: 276574 → 276574
[ 18.47s] Faces: 556132 → 556132
[ 18.59s] Površina: 206758.70 → 146757.23 mm²
[ 18.60s]
```

Slika 34-Rezultati Laplacian smoothing algoritma (PNG verzija)

```
[ 4.87s] ✓ Mreža: 216.924 verteksa, 430.246 trouglova
[ 4.92s] Površina: 155651.44 mm²
[ 4.95s]

[ 4.96s] Iteracije: 300, Relaksacija: 0.05
[ 11.02s] ✓ Smoothing završen | Površina: 155651.44 → 121142.77 mm²
[ 11.02s]
```

Slika 35- Marching Cubes i Laplacian (za DICOM fajlove)

Tokom implementacije, testirane su dve varijante algoritma za generisanje 3D mreže. Uočeno je da implementacija koronarne arterije (prosirena verzija i na ostale krvne sudove) proizvodi model sa manjim brojem poligona (430.246 trouglova) uz očuvanje anatomske tačnosti, dok druga varijanta generiše kompleksniju mrežu (556.132 trouglova) koja sadrži više geometrijskog šuma. **Modularni pristup je odabran kao preferirani**, jer nudi bolji balans između vizuelnog kvaliteta i računarske efikasnosti, a primena *Laplacian* zaglađivanja dodatno eliminiše artefakte nastale diskretizacijom volumena.

3.13. Vizualizacija 3D modela srca iz različitih perspektiva

Nakon generisanja 3D površinskog modela srca, sledeći korak u obradi predstavlja njegova vizualizacija iz više različitih perspektiva kako bi se omogućila detaljna analiza anatomske strukture. Funkcija `create_3d_renders` implementira proces kreiranja visokokvalitetnih rendera 3D mesh modela korišćenjem PyVista biblioteke u režimu renderovanja bez prikaza prozora (off-screen rendering).

Model se prikazuje iz četiri standardne projekcije: anteriorne (prednje), lateralne (bočne), superiorne (gornje) i kosog (oblique) pogleda, čime se obezbeđuje sveobuhvatan uvid u geometriju i oblik srčanog tkiva. Za svaki pogled podešava se pozicija kamere ili se koristi ručno definisana orijentacija u odnosu na centar modela, kako bi se obezbedila optimalna vidljivost strukture.

```

def create_3d_renders(mesh, output_dir, logger):
    """
    Kreiranje high-quality 3D renders iz različitih uglova
    """
    logger.log("\n [Vizualizacija] Kreiranje 3D rendera...")

    views = [
        ('anterior', 'xy', "Prednji pogled (Anterior)"),
        ('lateral', 'yz', "Bočni pogled (Lateral)"),
        ('superior', 'xz', "Gornji pogled (Superior)"),
        ('oblique', None, "Kosi pogled (Oblique)")
    ]

    for view_name, camera_pos, title in views:
        plotter = pv.Plotter(off_screen=True, window_size=config.RENDER_RESOLUTION)

        plotter.add_mesh(
            mesh,
            color=config.MESH_COLOR,
            smooth_shading=True,
            specular=0.8,
            specular_power=15,
            ambient=0.2,
            diffuse=0.7
        )

        if camera_pos:
            plotter.camera_position = camera_pos
        else:
            plotter.camera.position = (
                mesh.center[0] + 200,
                mesh.center[1] - 150,
                mesh.center[2] + 100
            )
            plotter.camera.focal_point = mesh.center

        plotter.background_color = config.BACKGROUND_COLOR

        plotter.add_scalar_bar(
            title="",
            n_labels=0,
            position_x=0.85,
            position_y=0.1
        )

        output_path = os.path.join(output_dir, f"04_render_{view_name}.png")
        plotter.show(screenshot=output_path)

        logger.log(f"    ✓ {title}: {output_path}")

```

Slika 36-Kreiranje 3D rendera

Renderovanje uključuje primenu materijalnih karakteristika poput sjaja, ambijentalnog i difuznog osvetljenja, što doprinosi realističnijem prikazu 3D modela. Svaki prikaz se zatim čuva kao visoko-rezolutna slika u izlaznom direktorijumu, što omogućava njegovu dalju upotrebu u izveštajima, prezentacijama ili medicinskoj analizi. Na ovaj način se obezbeđuje jasna i intuitivna interpretacija rekonstruisanog srčanog volumena iz različitih prostornih uglova.

```

[Vizualizacija] Kreiranje 3D rendera...
[ 29.93s]    ✓ Prednji pogled (Anterior): C:\Users\Korisnik\Downloads\archive (1)
\heart_3d_output\04_render_anterior.png
[ 30.46s]    ✓ Bočni pogled (Lateral): C:\Users\Korisnik\Downloads\archive (1)
\heart_3d_output\04_render_lateral.png
[ 30.91s]    ✓ Gornji pogled (Superior): C:\Users\Korisnik\Downloads\archive (1)
\heart_3d_output\04_render_superior.png
[ 31.66s]    ✓ Kosi pogled (Oblique): C:\Users\Korisnik\Downloads\archive (1)
\heart_3d_output\04_render_oblique.png
[ 31.66s]

```

Slika 37-Rezultati 3D rendera

Kako bi se potvrdila robustnost i prenosivost razvijenog algoritma, testiranje je izvršeno na više različitih skupova DICOM podataka. U svim testiranim slučajevima, algoritam je dosledno reprodukovao očekivane rezultate segmentacije srčanog volumena, uz minimalna odstupanja u preciznosti koja su direktna posledica varijacija u kvalitetu ulaznih snimaka (npr. različiti nivoi šuma ili intenzitet kontrasta).

3.14. Klinička vizualizacija 3D rekonstrukcije srca

Funkcija `create_clinical_view` implementira napredni klinički prikaz 3D rekonstrukcije srca kroz kombinaciju više vizuelnih modova u jedinstvenom prikazu. Cilj ove faze je da se obezbedi sveobuhvatna analiza srčanog volumena kroz integraciju 3D modela, preseka i volumetrijske vizualizacije u jednom koordinisanom prikazu.

Prvi panel prikazuje 3D rekonstrukciju srca sa osnovnim statističkim informacijama, uključujući izračunati volumen i površinu modela, čime se omogućava brza klinička procena veličine i geometrije strukture. Drugi i treći panel prikazuju aksijalni i sagitalni presek CT volumena, zajedno sa overlay maskom segmentiranog srčanog tkiva, što omogućava direktno poređenje originalnog intenziteta i rezultata segmentacije na karakterističnim anatomskim ravnima.

```
def create_clinical_view(volume, mask, mesh, metrics, output_dir, logger):
    """
    Profesionalni klinički prikaz sa multiple panelima
    """
    logger.log("\n [Vizualizacija] Kreiranje kliničkog pregleda...")

    plotter = pv.Plotter(
        shape=(2, 2),
        window_size=[2400, 2400],
        off_screen=True
    )

    plotter.subplot(0, 0)
    plotter.add_text(
        f"3D Rekonstrukcija Srca\n"
        f"Volumen: {metrics['volume_cm3']:.1f} cm³\n"
        f"Površina: {mesh.area/100:.1f} cm²",
        font_size=10,
        position='upper_left',
        color='black'
    )
    plotter.add_mesh(
        mesh,
        color=config.MESH_COLOR,
        smooth_shading=True,
        specular=0.7
    )
    plotter.camera_position = 'xy'
    plotter.background_color = 'white'

    plotter.subplot(0, 1)
    plotter.add_text(
        "Axial Presek + Segmentacija",
        font_size=10,
        position='upper_left',
        color='black'
    )

    mid_z = volume.shape[2] // 2

    slice_plane = pv.ImageData()
    slice_plane.dimensions = (volume.shape[0], volume.shape[1], 1)
    slice_plane.spacing = (1, 1, 1)
```

Slika 38- Kombinovani prikaz CT preseka i 3D rekonstrukcije srca(prvi deo)

Četvrti panel prikazuje kombinaciju volumetrijskog renderovanja CT podataka i transparentnog 3D modela srca, čime se dobija kompletan prostorni uvid u odnos između sirovih podataka i rekonstruisane strukture. Sinhronizacija svih prikaza omogućena je povezivanjem kamera, što obezbeđuje konzistentnu perspektivu tokom analize.

```

slice_sag.point_data["HU"] = volume[mid_x, :, :].flatten(order="F")

plotter.add_mesh(slice_sag, cmap="bone", clim=[-200, 300])

if np.any(mask[mid_x, :, :]):
    mask_sag = pv.ImageData()
    mask_sag.dimensions = (mask.shape[1], mask.shape[2], 1)
    mask_sag.spacing = (1, 1, 1)
    mask_sag.point_data["mask"] = mask[mid_x, :, :].astype(float).flatten(order="F")
    plotter.add_mesh(mask_sag, cmap="Reds", opacity=0.5)

plotter.view_yz()
plotter.background_color = 'white'

plotter.subplot(1, 1)
plotter.add_text(
    "Volume Rendering + Overlay",
    font_size=10,
    position='upper_left',
    color='black'
)

grid = pv.ImageData()
grid.dimensions = np.array(volume.shape)
grid.spacing = (1, 1, 1)
grid.point_data["HU"] = volume.flatten(order="F")

plotter.add_volume(
    grid,
    cmap="bone",
    opacity="sigmoid",
    opacity_unit_distance=2.0,
    shade=True
)

plotter.add_mesh(
    mesh,
    color='red',
    opacity=0.4,
    smooth_shading=True
)

plotter.camera_position = 'xy'
plotter.background_color = 'white'

plotter.link_views()

output_path = os.path.join(output_dir, "05_clinical_multipanel.png")
plotter.show(screenshot=output_path)

logger.log(f" ✓ Klinički prikaz sačuvan: {output_path}")

```

Slika 39- Kombinovani prikaz CT preseka i 3D rekonstrukcije srca(drugi deo)

Ovakav multipanelni pristup predstavlja klinički orijentisanu vizualizaciju koja integriše različite nivoe informacija (2D preseke i 3D model), omogućavajući bolju interpretaciju anatomskih odnosa i validaciju segmentacionih rezultata u medicinskoj praksi.

```

[Vizualizacija] Kreiranje kliničkog pregleda...
[ 36.33s] ✓ Klinički prikaz sačuvan: C:\Users\Korisnik\Downloads\archive (1)
\heart_3d_output\05_clinical_multipanel.png
[ 36.36s]

```

Slika 40 –Rezultat komandi(multiprikazni panel)

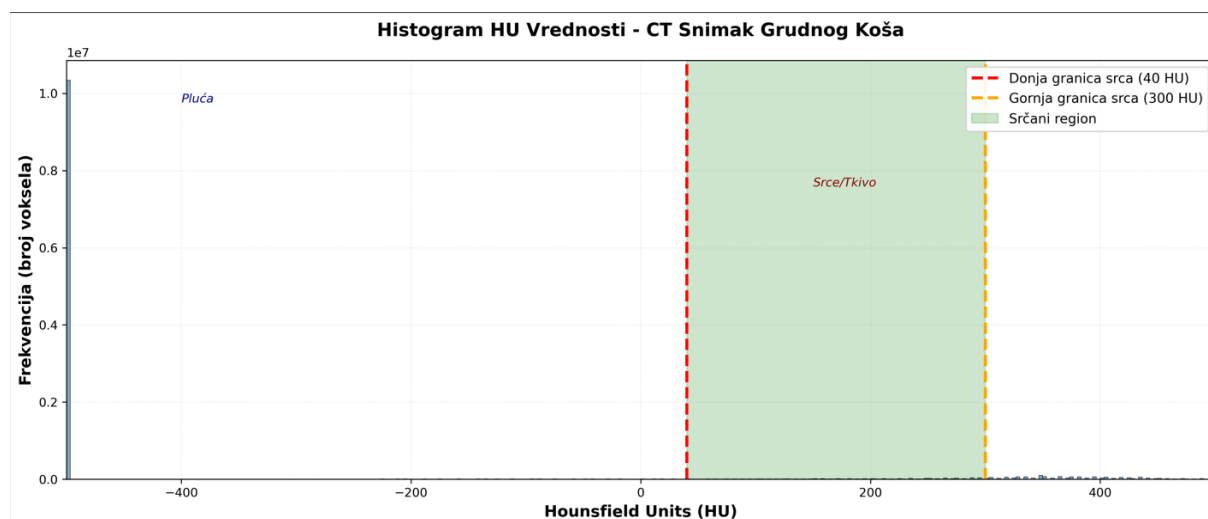
Tokom validacije sistema, primenjen je diferencijalni pristup vizuelizaciji prilagođen anatomskom fokusu: za **koronarne arterije** korišćen je širi HU opseg $[-200, 500]$, neophodan za izolaciju visokodenzitetnih struktura poput kontrasta i kalcifikacija, dok je za **celokupan srčani volumen** optimizovan uži opseg $[-200, 300]$ radi postizanja optimalnog kontrasta miokarda i jasnije morfološke diferencijacije. Analiza je ponovljena na heterogenim skupovima DICOM podataka, pri čemu je algoritam dosledno reprodukovao rezultate, potvrđujući svoju robustnost i nezavisnost od varijacija u kvalitetu ulaznih snimaka. Ovakva generalizacija potvrđuje da je razvijeno rešenje primenjivo u realnim kliničkim scenarijima sa visokim stepenom pouzdanosti.

4.REZULTATI

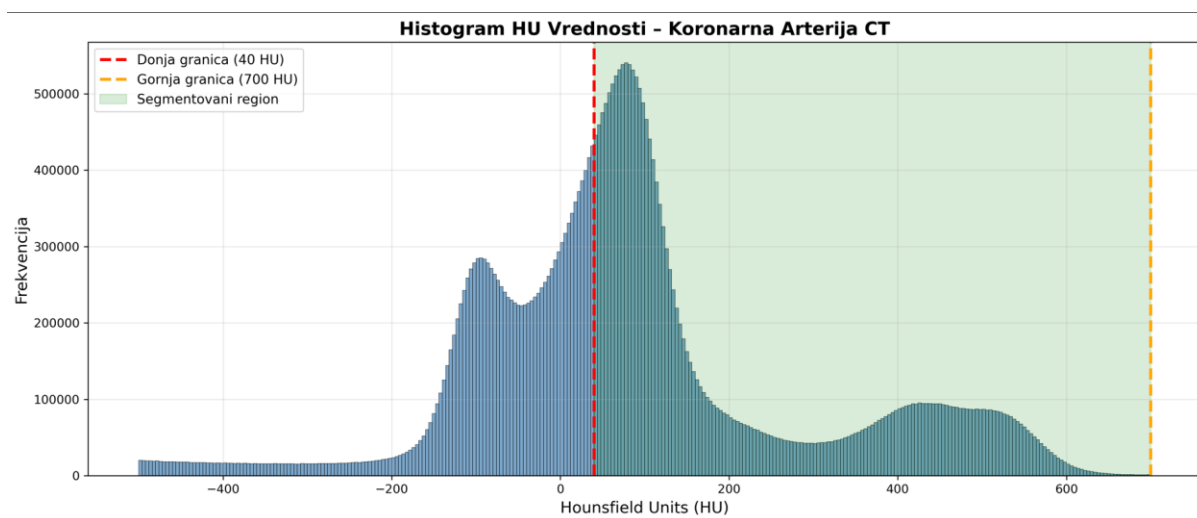
4.1.Izgled histograma

Histogram HU vrednosti sa ucrtanim opsegom od **40 do 300 HU** pokazuje da je prag postavljen u domenu mekih tkiva, ciljano izolujući srčani mišić i krvne sudove. Na prikazu je jasno definisana donja granica koja počinje iznad nivoa vode i masti, čime se srčani region efikasno odvaja od okolnog perikardijalnog masnog tkiva i plućnog parenhima koji zauzimaju znatno niže vrednosti.

Gornja granica od **300 HU** osigurava da se proces segmentacije zaustavi pre domena visokih gustina, što omogućava jasno razdvajanje mekog tkiva srca od koštanih struktura grudnog koša, poput rebara i pršljenova. Ovakav izbor praga omogućava softveru da precizno izoluje anatomiju srca, dok istovremeno filtrira vazduh iz pluća i tvrdu kortikalnu kost koji bi se nalazili na suprotnim krajevima prikazanog histograma.



Slika 41-Histogram HU vrednosti CT snimka



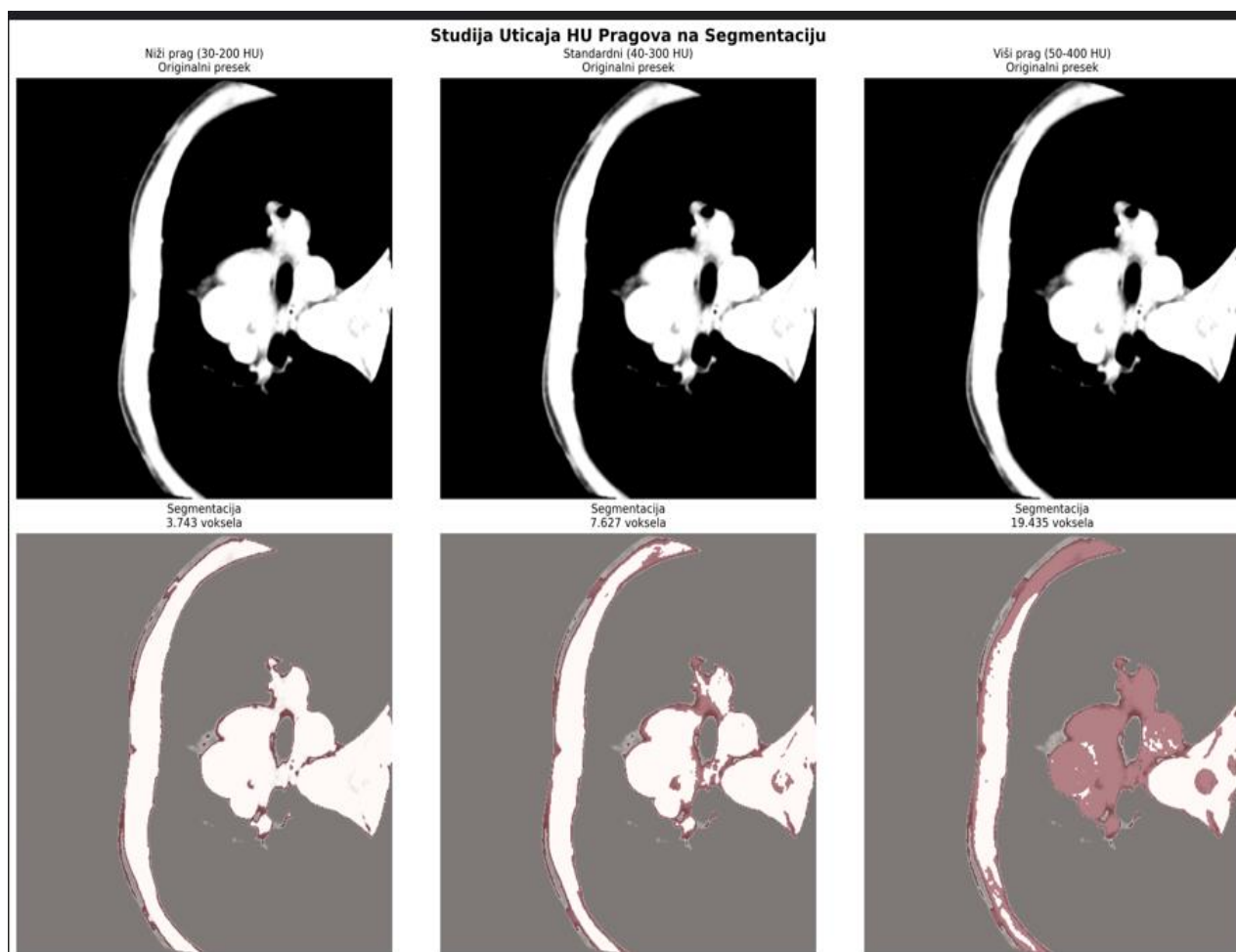
Slika 42- Histogram distribucije Hounsfieldovih (HU) vrednosti sa označenim opsegom segmentacije [40, 700] HU, koji ilustruje postupak izolacije arterijskog lumena eliminacijom tkiva niske gustine i šuma.

Dok su **PNG podaci** (Slika 32]) efikasni za brzu vizuelizaciju i testiranje algoritama, **DICOM podaci** (Slika 33) pružaju neophodnu fizičku dubinu Hounsfieldove (HU) skale. Razlika leži u tome što DICOM zadržava 12-bitnu/16-bitnu preciznost, omogućavajući postavljanje pragova (40–700 HU) na osnovu realnih karakteristika tkiva, dok PNG usled 8-bitne kompresije gubi te metapodatke. Stoga, PNG služi za brzi pregled, dok je DICOM nezaobilazan za visoku anatomska preciznost rekonstrukcije koronarnih arterija.

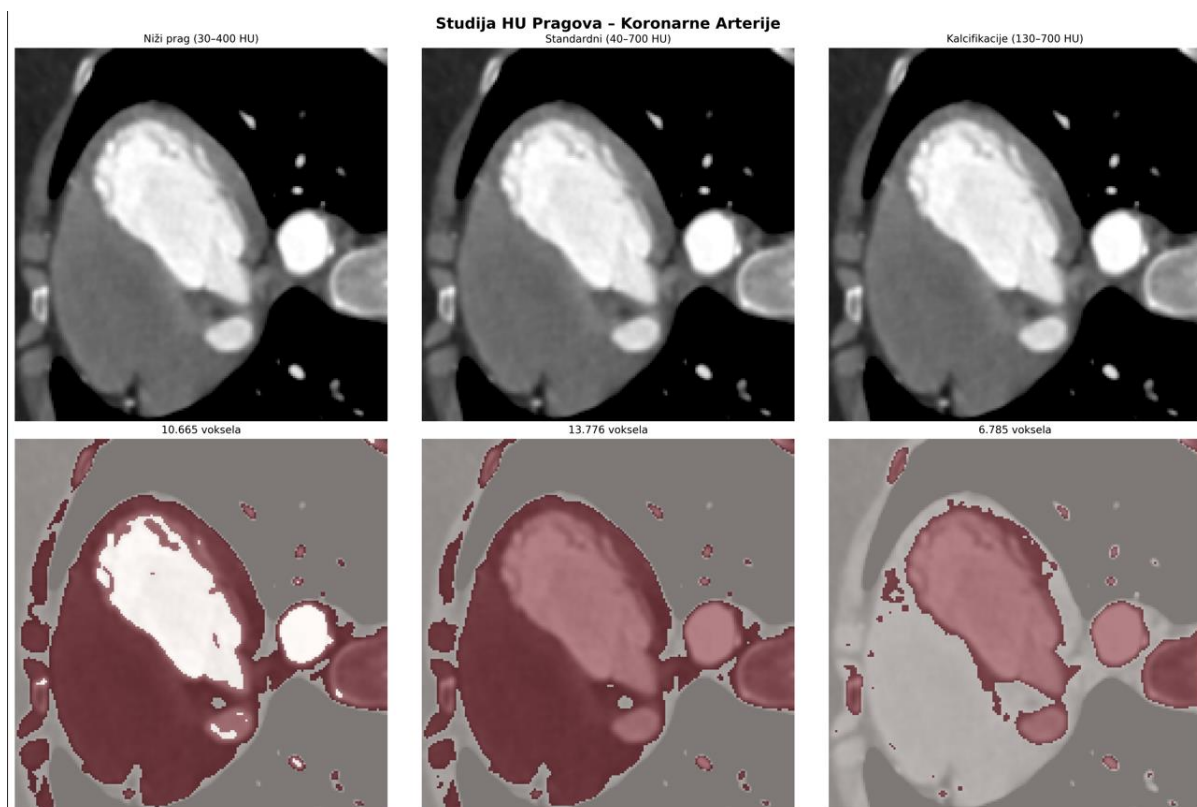
4.2.Uticaj HU pragova na segmentaciju

Ova studija prikazuje kako promena opsega Hounsfieldovih jedinica menja obuhvat tkiva na CT snimku. Poređenjem tri nivoa praga uočava se direktna korelacija: sa širenjem opsega (od nižeg ka višem), broj obuhvaćenih voksela drastično raste, što se vizuelno manifestuje većom površinom crveno-markiranih regija.

Standardni prag (40–300 HU) pruža optimalan balans za izolaciju mekih tkiva. Nasuprot tome, **viši prag (50–400 HU)** uzrokuje prekomernu segmentaciju koja uključuje i gušće strukture, dok **niži prag (30–200 HU)** ostaje previše restriktivan, izostavljajući bitne delove anatomije. Ovi prikazi dokazuju da je precizno podešavanje HU granica ključno za tačno razdvajanje organa od okoline.



Slika 43-HU pragovi i segmentacija

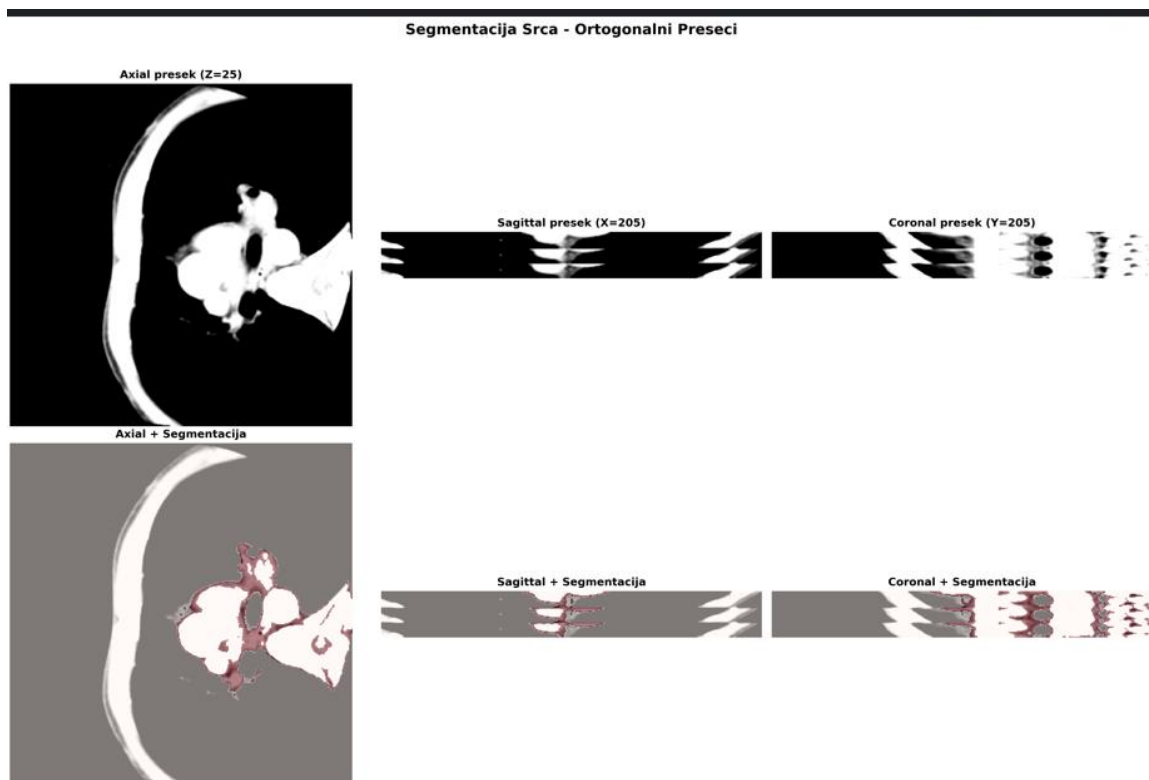


Slika 44- Uticaj HU pragova na segmentaciju koronarnih arterija

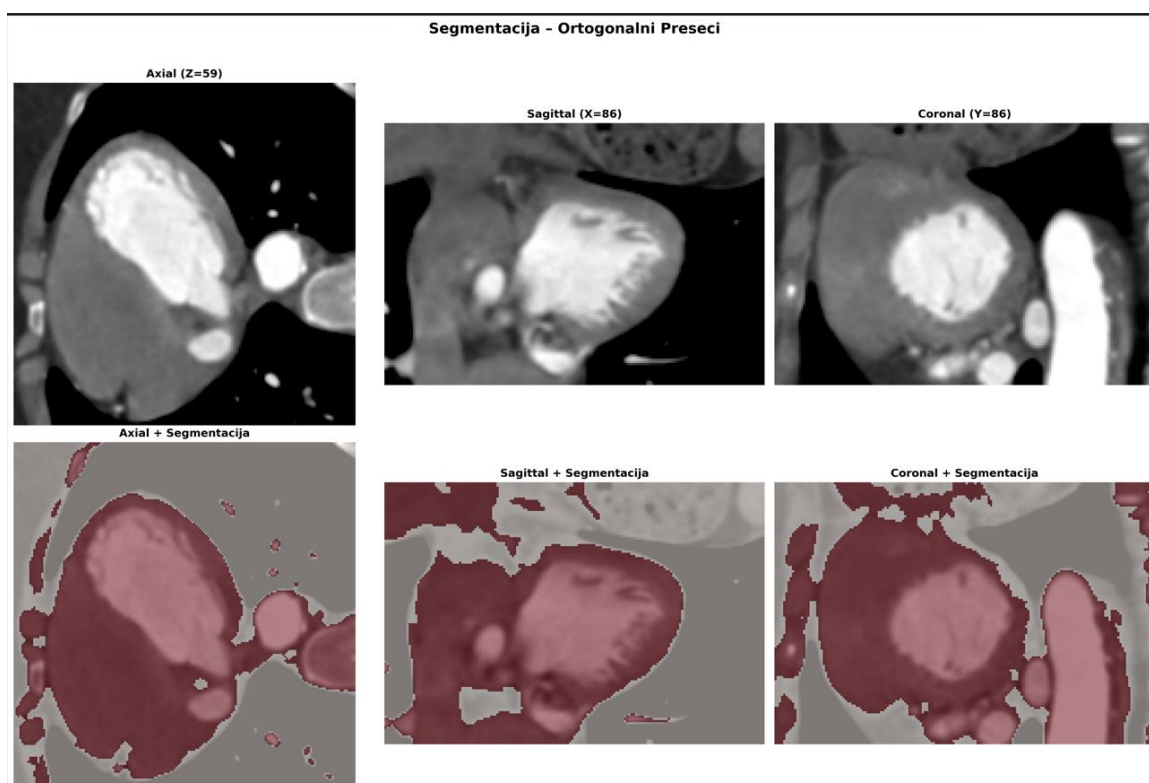
Poređenje DICOM i PNG obrade pokazuje da direktna manipulacija DICOM podacima omogućava precizniju kalibraciju HU pragova (Slika 34), što je ključno za tačnu segmentaciju koronarnih arterija. Dok PNG format pruža bržu obradu, on gubi izvorne fizičke informacije tkiva, što dovodi do veće osetljivosti na artefakte. Studija osetljivosti (30–400 HU) potvrđuje da precizna kontrola praga (Standardni 40–300 HU) direktno optimizuje broj detektovanih voksela, osiguravajući visoku specifičnost anatomske rekonstrukcije koju PNG pristup ne može u potpunosti da replikuje.

4.3.Pregled preseka

Ova slika prikazuje rezultate segmentacije srca kroz tri osnovne ravni — **aksijalnu** ($Z=25$), **sagitalnu** ($X=205$) i **koronalnu** ($Y=205$), čime se dobija kompletan trodimenzionalni uvid u preciznost primenjenog HU praga. Gornji red panela prikazuje originalne radiološke snimke na specifičnim koordinatama, dok donji red ilustruje segmentisane regione označene crvenom bojom, potvrđujući dosledno izdvajanje ciljanog mekog tkiva u svim ravnima. Ovakav ortogonalni prikaz je ključan za verifikaciju prostornog kontinuiteta segmentacije, jer osigurava da izolovani volumen srca zadržava anatomska tačnost bez obzira na ugao posmatranja, istovremeno jasno razgraničavajući srčane strukture od okolnog tkiva i skeleta u 3D prostoru.



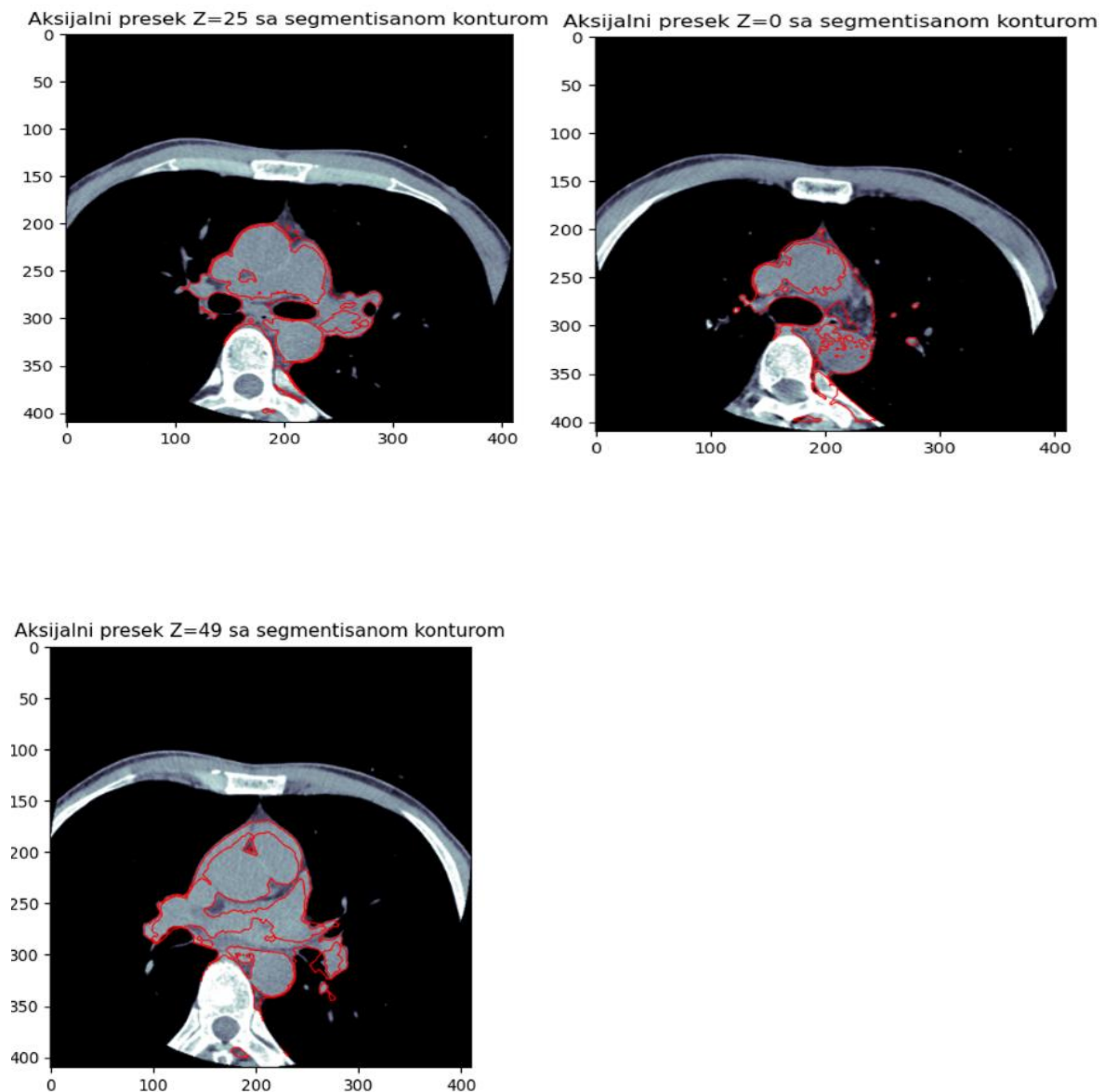
Slika 45-Ortogonalni preseci



Slika 46- Multiravanska vizuelizacija segmentacionih maski koronarne arterije

Poređenje DICOM i PNG obrade (Slike 34 i 35) potvrđuje da PNG format usled kompresije gubi izvorne metapodatke, što dovodi do prekida anatomskog kontinuiteta u sagitalnoj i koronalnoj ravni. Dok DICOM obezbeđuje preciznu multiravansku rekonstrukciju (MPR) i validnost 3D geometrije, PNG prikaz je nepouzdan za medicinsku analizu zbog gubitka prostorne rezolucije. Stoga je direktna obrada DICOM podataka neophodna za visoku specifičnost segmentacije koronarnih arterija.

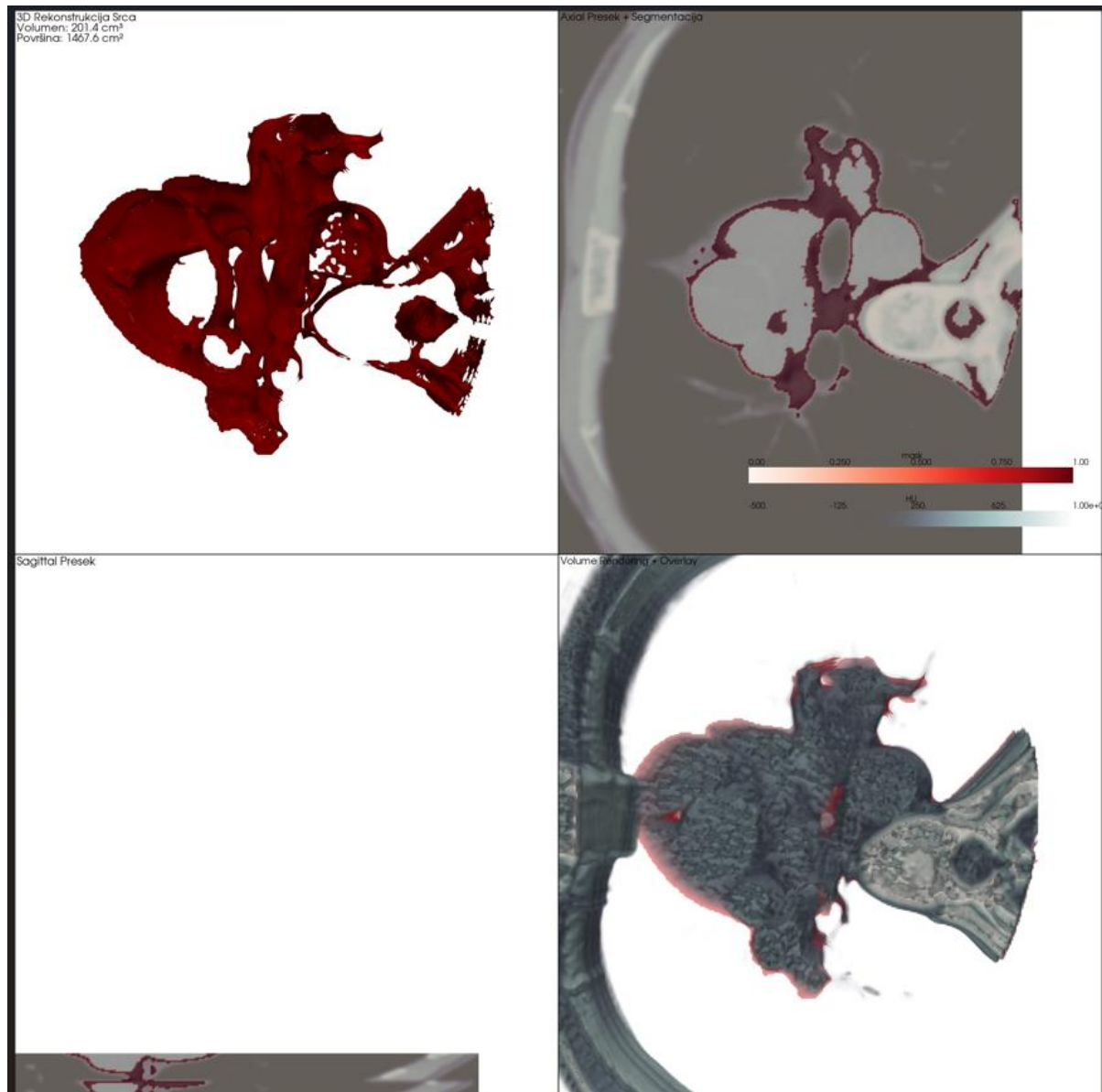
4.4. Aksijalni preseki



Slika 47-Razlicite vrste aksijalnih preseka

Slike prikazuju aksijalne preseke CT snimka na različitim dubinama (Z=0, Z=25, Z=49) sa precizno iscrtanim crvenim konturama koje definišu segmentisani region srca. Ove konture vizuelno potvrđuju da izabrani HU prag dosledno prati anatomsku granicu mekog tkiva kroz različite slojeve grudnog koša.

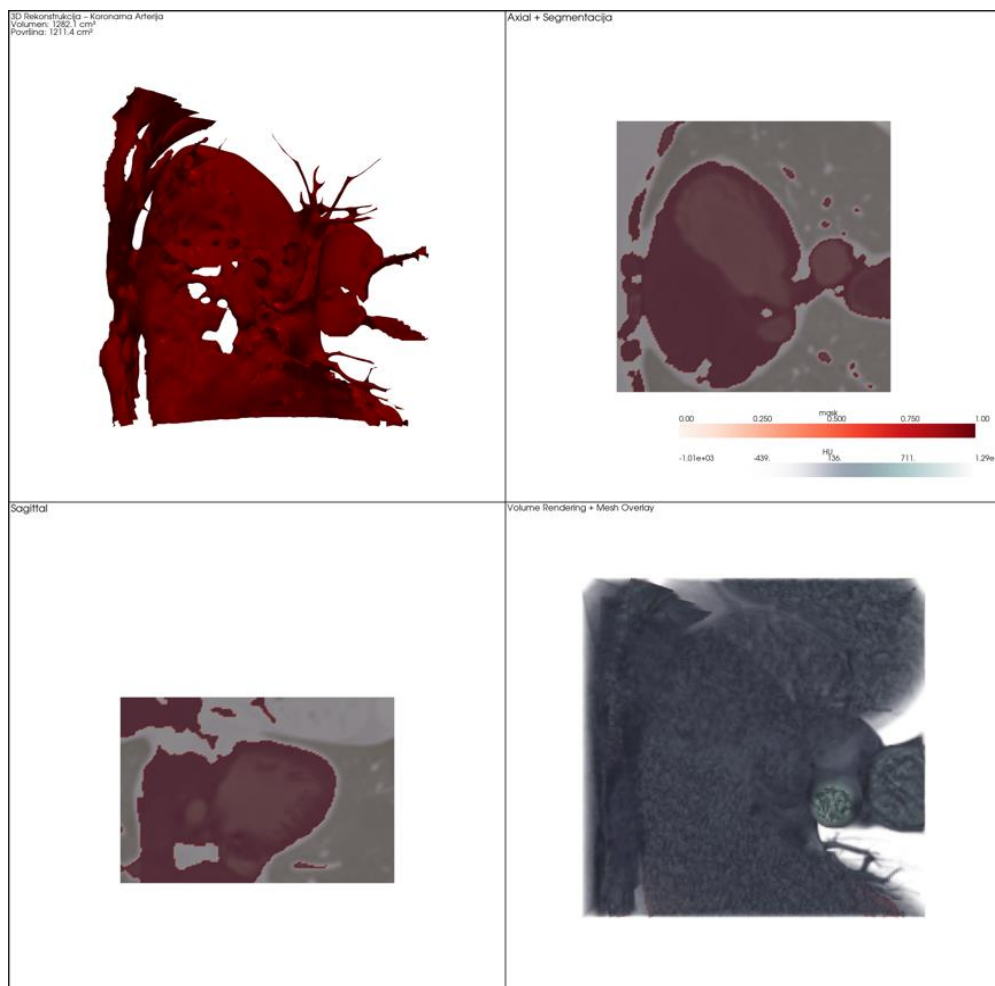
4.5. Trodimenzionalna rekonstrukcija i volumetrijska analiza



Slika 48- Rezultati 3D rekonstrukcije i volumetrijske analize segmentisanog srca

Ovaj prikaz objedinjuje finalni **3D model srca** sa pratećim kvantitativnim parametrima, poput zapremine od **201.4 cm³** i površine od **1467.6 cm²**, dobijenih nakon procesa segmentacije. Paneli prikazuju integraciju segmentisanih kontura sa originalnim CT

podacima kroz *Volume Rendering* i ortogonalne preseke, čime se potvrđuje anatomska preciznost i prostorni kontinuitet rekonstruisanog modela.



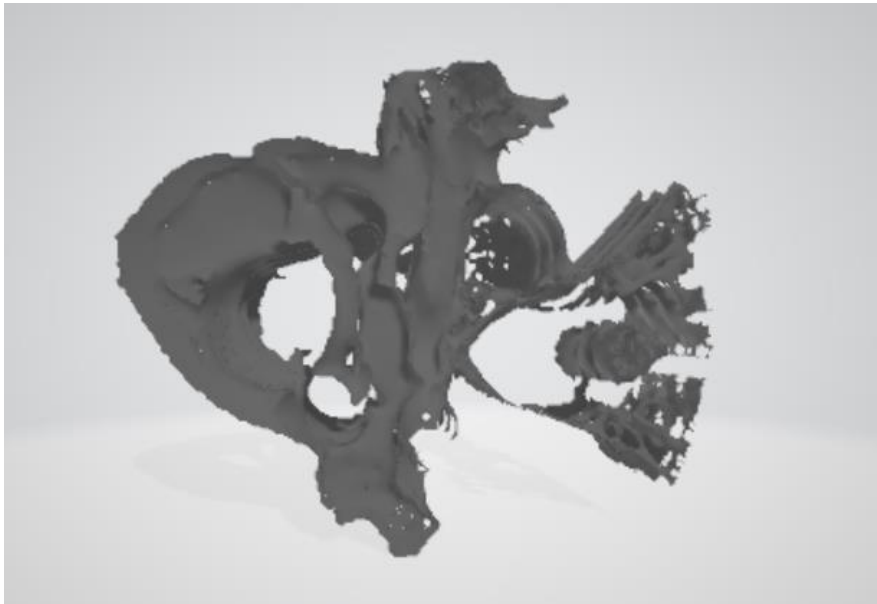
Slika 49- Prikaz rezultata segmentacije i 3D rekonstrukcije koronarne arterije iz DICOM podataka

Slika prikazuje rezultate 3D rekonstrukcije koronarne arterije, dobijene obradom DICOM podataka primenom *scikit-image*, *scipy*, *numpy* i *pyvista* biblioteka. Gornji levi panel prikazuje finalni model sa izračunatim volumenom od 1282.1 cm³ i površinom od 1211.4 cm², dok ostali paneli (aksijalni, sagitalni i *volume rendering*) služe za verifikaciju segmentacije i analizu anatomske strukture u odnosu na izvorni signal.

4.6.3D rekonstrukcija i renderi

Ova slika prikazuje izolovanu geometrijsku mrežu (mesh) srca eksportovanu u **STL formatu**, koji predstavlja standard za digitalnu fabriku i 3D štampu. Model je prikazan u monohromatskom modu kako bi se jasno uočila topologija površine, očuvani anatomske detalji i složenost unutrašnjih šupljina dobijenih procesom automatizovane segmentacije. Ovi

podaci služe kao osnova za fizičku rekonstrukciju organa ili dalju biomaterijalnu simulaciju u inženjerskom okruženju.



Slika 50- Finalni STL model srca spreman za 3D štampu

Sledeće slike prikazuju finalni proces **3D renderovanja i vizuelizacije rekonstruisanog srca** iz različitih perspektiva, naglašavajući kompleksnost unutrašnje geometrije i kontinuitet vaskularnih struktura. Korišćenjem naprednih tehnika senčenja i mapiranja tekstura, model verno reprodukuje anatomske volumene izolovane tokom segmentacije, omogućavajući detaljan uvid u prostorne odnose srčanih komora i pretkomora. Ovakva visokokvalitetna vizuelizacija ključna je za finalnu validaciju modela pre njegove primene u medicinskim simulacijama ili digitalnoj fabrikaciji.



Slika 51- Bočni prikaz



Slika 52-Prednji prikaz



Slika 53-Pogled odozgo(gornji prikaz)

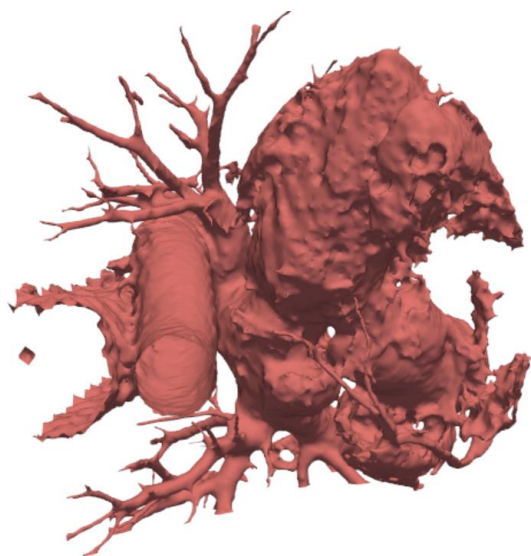


Slika 54-Kosi prikaz(kosi pogled)pod uglom

Dodatno(3D prikaz koronarne arterije):



Slika 55-3D model koronarne arterije



Slika 56-Pyvista 3D model koronarne arterije



Slika 57-Kosi pogled koronarna arterija



Slika 58-Bocni prikaz koronarne arterije



Slika 59-Pogled odozgo



Slika 60-Pogled sa prednje strane

5.ZAKLJUCAK

U radu je prikazan kompletan, ponovljiv i praktičan inženjerski tok koji iz sirove CT DICOM serije generiše precizan 3D model srčanog mišića. Ključni koraci procesa obuhvatili su: normalizaciju Hounsfieldovih jedinica (HU), izotropno resamplovanje volumena, te pažljiv izbor optimalnog praga segmentacije (40–300HU) zasnovan na analizi histograma gustine tkiva. Kroz primenu morfoloških operacija i algoritama za popunjavanje rupa, uspešno je izolovan kontinualni volumen srca, dok je finalna ekstrakcija površine izvršena *marching-cubes* metodom uz primenu Taubin glačanja radi postizanja visokog kvaliteta meša.

Formirani proces omogućio je stabilnu generaciju STL modela spremnog za dalju upotrebu, dok prateće vizuelizacije — uključujući studiju uticaja HU pragova, ortogonalne preseke u tri ravni i finalne rendere iz različitih perspektiva (*anterior, superior, lateral, oblique*) — pružaju transparentnu kontrolu kvaliteta nad svakom fazom obrade. Poseban fokus stavljen je na preciznost kontura i volumetrijsku analizu, čime je dobijena zapremina modela od 201.4 cm³, što potvrđuje anatomsku realističnost rekonstrukcije.

Ograničenja rada proističu iz prirode medicinskih snimaka, gde šum i lokalne varijacije u gustini tkiva mogu uticati na preciznost ivica, dok rezolucija snimka postavlja granicu detaljnosti rekonstrukcije malih krvnih sudova. Ipak, primenjeni filteri i algoritmi za optimizaciju meša značajno su ublažili ove efekte, očuvavši pri tome integritet anatomske oblika.

Zaključno, predloženi tok pouzdano prevodi sirove radiološke podatke u upotrebljiv digitalni 3D model srca, čime su svi ciljevi istraživanja ispunjeni, a postavljena je i čvrsta osnova za potencijalnu fizičku fabrikaciju putem 3D štampe ili dalja istraživanja u oblasti bioinženjeringa.

6.LITERATURA

- [1] I. Bankman, *Handbook of Medical Image Processing and Analysis*, 2nd ed. New York, NY, USA: Academic Press, 2008.
- [2] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3D surface construction algorithm," in *Proc. 14th Annu. Conf. Comput. Graph. Interact. Techn. (SIGGRAPH '87)*, 1987, pp. 163–169.
- [3] G. Taubin, "A signal processing approach to fair surface design," in *Proc. 22nd Annu. Conf. Comput. Graph. Interact. Techn. (SIGGRAPH '95)*, 1995, pp. 351–358.
- [4] O. S. Pianykh, *Digital Imaging and Communications in Medicine (DICOM): A Practical Introduction and Survival Guide*, 2nd ed. Berlin, Germany: Springer-Verlag, 2011.
- [5] J. T. Bushberg, J. A. Seibert, E. M. Leidholdt, and J. M. Boone, *The Essential Physics of Medical Imaging*, 3rd ed. Philadelphia, PA, USA: Lippincott Williams & Wilkins, 2011.
- [6] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, et al., "scikit-image: image processing in Python," *PeerJ*, vol. 2, p. e453, 2014. [Online]. Available: <https://doi.org/10.7717/peerj.453>
- [7] B. C. Lowekamp, D. T. Chen, L. Ibáñez, and D. Blezek, "The Design of SimpleITK," *Frontiers in Neuroinformatics*, vol. 7, Art. no. 45, Dec. 2013, doi: 10.3389/fninf.2013.00045.

- [8] B. Sullivan and A. Kaszynski, "PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK)," *Journal of Open Source Software*, vol. 4, no. 37, p. 1450, May 2019, doi: 10.21105/joss.01450.
- [9] D. Mason et al., "pydicom: An open source DICOM library," 2023. [Online]. Available: <https://github.com/pydicom/pydicom> (Zenodo DOI available per version).
- [10] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.
- [11] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.
- [12] B. Rim, S. Kim, and H. Lee, "Semantic Cardiac Segmentation in Chest CT Images Using K-Means Clustering and the Mathematical Morphology Method," *Sensors*, vol. 21, no. 8, p. 2675, 2021, doi: 10.3390/s21082675.
- [13] W. K. Cheung et al., "A Computationally Efficient Approach to Segmentation of the Aorta and Coronary Arteries Using Deep Learning," *IEEE Access*, vol. 9, pp. 108873–108888, 2021, doi: 10.1109/ACCESS.2021.3099030.
- [14] D. Hinck et al., "Automatic Segmentation of Cardiovascular Structures on Chest CT Data Sets: An Update of the TotalSegmentator," *European Journal of Radiology*, vol. 185, 2025.

